

# The Pinnacle Architecture: Reducing the cost of breaking RSA-2048 to 100 000 physical qubits using quantum LDPC codes

Paul Webster,<sup>\*</sup> Lucas Berent, Omprakash Chandra, Evan T. Hockings,  
Nouédyne Baspin, Felix Thomsen, Samuel C. Smith, and Lawrence Z. Cohen<sup>†</sup>  
*Iceberg Quantum, Sydney*

The realisation of utility-scale quantum computing inextricably depends on the design of practical, low-overhead fault-tolerant architectures. We introduce the *Pinnacle Architecture*, which uses quantum low-density parity check (QLDPC) codes to allow for universal, fault-tolerant quantum computation with a spacetime overhead significantly smaller than that of any competing architecture. With this architecture, we show that 2048-bit RSA integers can be factored with less than one hundred thousand physical qubits, given a physical error rate of  $10^{-3}$ , code cycle time of  $1\text{ }\mu\text{s}$  and a reaction time of  $10\text{ }\mu\text{s}$ . We thereby demonstrate the feasibility of utility-scale quantum computing with an order of magnitude fewer physical qubits than has previously been believed necessary.

## I. INTRODUCTION

Quantum computers offer the promise of efficient solutions to currently intractable problems with the potential to allow breakthroughs in areas such as cryptography [1], materials science and chemistry [2]. However, due to the precision required and the significant levels of noise that afflict all engineered quantum systems, this is only possible if quantum computers are fault tolerant [3]. Fault-tolerant quantum architectures are therefore a cornerstone of all efforts to build useful quantum computers.

Sophisticated fault-tolerant architectures have been developed based on the surface code [4–8]. However, these suffer from a very high overhead, since they require hundreds or thousands of physical qubits to encode a single logical qubit with a low enough failure rate to allow utility-scale computations. As a result, utility-scale quantum computers based on such architectures are expected to require at least one million physical qubits [8, 9]. Scaling quantum hardware to this size poses formidable challenges [9]. Developing fault-tolerant quantum architectures with lower overhead is therefore of the utmost importance.

To meet this goal, we introduce the *Pinnacle Architecture*. This architecture achieves substantial spacetime reductions compared with prior state-of-the-art architectures [8, 10] by reducing the space overhead relative to surface code architectures without a commensurate increase in time overhead. We realise these savings through the use of processing units constructed from bridged QLDPC code blocks equipped with modular, efficient gadgets for performing gates by generalised surgery [11]. We also introduce a new component—the *magic engine*—which exploits the multiple logical qubits of QLDPC codes to simultaneously support magic state distillation and injection in a single code block, allowing for constant throughput of high-fidelity magic states with low over-

head. Moreover, we introduce *Clifford frame cleaning* as a new method that allows for efficient parallelism of operations across processing units. In particular, this allows for parallel access to a quantum memory that can allow for spacetime overhead reductions by enabling algorithms to be parallelised by duplicating processing units while keeping only a single memory. Scalability and hardware-compatibility is also ensured through a modular structure which ensures that connectivity between physical qubits is only required on a length scale constant in the number of logical qubits. We summarise the main features of the architecture in Section II A and, after reviewing relevant background concepts in Section III, we then provide a full presentation of the general architecture Section IV, and a specific instantiation in Section V.

We benchmark the performance of the Pinnacle Architecture by presenting a compilation to a standard application: factoring 2048-bit RSA integers [7, 8]. With standard hardware assumptions (i.e., a physical error rate of  $10^{-3}$ , code cycle time of  $1\text{ }\mu\text{s}$  and reaction time of  $10\text{ }\mu\text{s}$ ), we show that factoring can be achieved with fewer than one hundred thousand physical qubits. This outperforms the previous best result by an order of magnitude [8]. We further show the broad applicability of the architecture by showing that it allows for classically intractable instances of the problem of determining the ground-state energy of the Fermi-Hubbard model to be achieved with tens of thousands of physical qubits, under the same assumptions. Again, this amounts to an order-of-magnitude improvement on the best prior end-to-end resource estimates [12]. In addition to these results, we also apply our compilations under alternative hardware assumptions to provide optimised resource estimates applicable across a range of hardware platforms. We summarise these results in Section II B and present further details in Section VI.

Through the Pinnacle Architecture, we thus open the door to utility-scale quantum computing on one hundred thousand physical qubit devices. This has the potential to significantly accelerate the timescale for commercialised and impactful quantum computers.

<sup>\*</sup> paul@iceberg-quantum.com

<sup>†</sup> larry@iceberg-quantum.com

## II. SUMMARY OF CONTRIBUTIONS

### A. Pinnacle Architecture

The Pinnacle Architecture consists of:

- *Processing Units* consisting of bridged processing blocks each constructed from a QLDPC code block with an ancillary measurement gadget system [11]. An arbitrary logical Pauli product measurement can be performed on the logical qubits of the unit in each logical cycle.
- *Magic Engines* consisting of a QLDPC code block along with ancillary systems for injecting noisy magic states. In each logical cycle, the magic engine stores a high-fidelity magic state as it is injected into a processing unit in parallel with hosting magic state distillation to prepare a high-fidelity magic state for the next logical cycle. It thereby provides a high-fidelity magic state per logical cycle for each processing unit to allow for universal quantum computing.
- *Memory* included as an optional component that allows for especially low overhead quantum storage in code blocks, which can be accessed by processing units via ports.

Figure 1 shows how the Pinnacle Architecture is assembled from these modules.

Compilation is performed via Pauli-based computation [13]. This allows for universal fault-tolerant quantum computing on arbitrarily many logical qubits with a time cost that scales with the  $T$  count.

Features of the Pinnacle Architecture include:

- *Low Spacetime Overhead:* The use of QLDPC codes allows order-of-magnitude reductions in physical qubit number compared with surface code architectures. This is achieved without the corresponding increase in time overhead of the previous state-of-the-art QLDPC code architecture—the *bicycle architecture* of Ref. [10]—through the use of efficient gadget systems that allow for arbitrary logical Pauli measurements instead of only a subset of them. It therefore substantially reduces the total spacetime overhead compared to previous fault-tolerant architectures.
- *Limited Connectivity and Routing:* The architecture requires only interactions between physical qubits on the scale of a processing block, which is constant in the number of logical qubits. This means that it does not depend on all-to-all connectivity but instead is implementable on hardware platforms that support quasi-local connections between physical qubits separated by a bounded distance. Moreover,

arbitrary quantum circuits can be performed efficiently using a static configuration of processing blocks without depending on long-distance routing.

- *Modularity and Parallelism:* Computations can be separated across multiple processing units with limited connectivity between them, allowing compatibility with modular hardware. Efficient parallelism of non-Clifford gates is also supported by introducing the new technique of Clifford frame cleaning to supplement this modular structure. This technique also allows parallel, read-only access to memory by multiple processing units.

The Pinnacle architecture therefore offers a new alternative architecture that is both practical for implementation on a range of hardware platforms and significantly more efficient than previous state-of-the-art alternatives.

### B. Results

For concreteness, we present a specific instantiation of the Pinnacle Architecture using the family of generalised bicycle codes and the modular, efficient measurement gadgets introduced in Ref. [11]. Based on compilation to this instantiation, we present resource estimates for two benchmark applications.

First, we consider determination of the ground state energy of the Fermi-Hubbard model via plaquette Trotterisation [14]. As shown in Fig. 2, we achieve order-of-magnitude reductions in physical qubit number relative to the best available end-to-end surface code analysis [12]. For example, we find that at a lattice size of  $L = 16$  (with a coupling strength of  $u/\tau = 4$ ), only 62 thousand physical qubits are required at a physical error rate of  $p = 10^{-3}$  and only 22 thousand at  $p = 10^{-4}$ . This compares with 940 thousand and 200 thousand, respectively, in Ref. [12]. We achieve these results while maintaining a modest runtime per shot of 1–4 minutes with microsecond code cycle times or 1–3 days with millisecond code cycle times.

Second, we analyse factoring RSA integers using an algorithm based on that of Ref. [8]. Fig. 3 shows the required physical qubits and runtime to factor a 2048-bit integer for different code cycle times and physical error rates. With standard assumptions of a physical error rate of  $p = 10^{-3}$  and a code cycle time of 1  $\mu$ s and a 10  $\mu$ s reaction time (see Section III for definitions of these timescales), factoring is possible with fewer than one hundred thousand physical qubits, compared with the previous best result of close to one million physical qubits [8]. Moreover, through parallelising the algorithm, we achieve an efficient spacetime trade-off that allows for low overhead factoring in feasible runtimes even with longer code cycle times. For example, with a code cycle time of 1 ms, factoring can be completed in one month with 3.1 million physical qubits at a physical error rate of  $p = 10^{-4}$  (typical of trapped ions [15]) or with 13 million physical qubits at a physical error rate of  $p = 10^{-3}$  (typical of, for example, neutral atoms [16]).

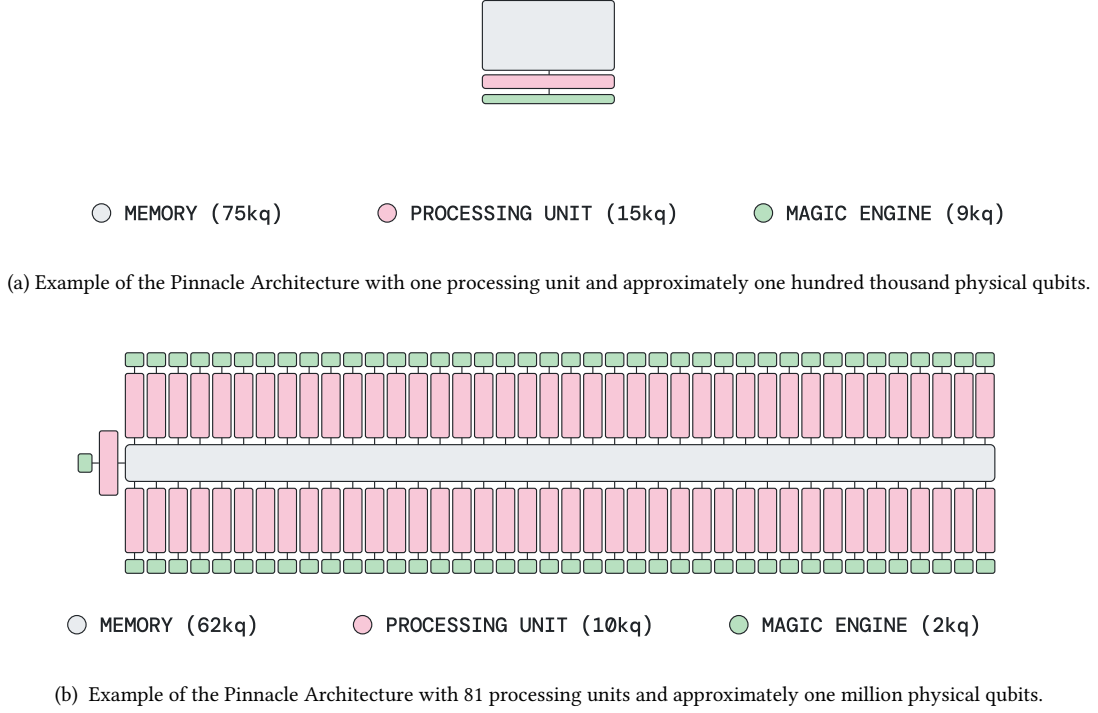


FIG. 1. Examples of the Pinnacle Architecture. These two examples represent specific examples of different space-time trade-offs and code block choices, optimised for RSA-2048 factoring in different hardware regimes. Example (a) allows factoring in one month with a physical error rate of  $p = 10^{-3}$  and a code cycle time of  $t_c = 1 \mu\text{s}$ . Example (b) allows factoring in three months with a physical error rate of  $p = 10^{-4}$  and a code cycle time of  $t_c = 1 \text{ ms}$ . Shorter runtimes can be achieved by adding more processing units, increasing parallelisation at the cost of additional physical qubits.

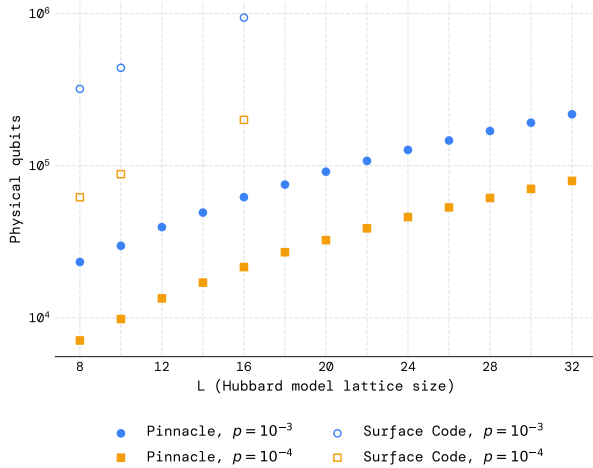


FIG. 2. Physical qubits required for determining the ground state energy of the Fermi-Hubbard model on an  $L \times L$  lattice to 0.5% relative precision. Surface code values correspond to the minimum quoted number of physical qubits with  $u/\tau = 4$  in Ref. [12].

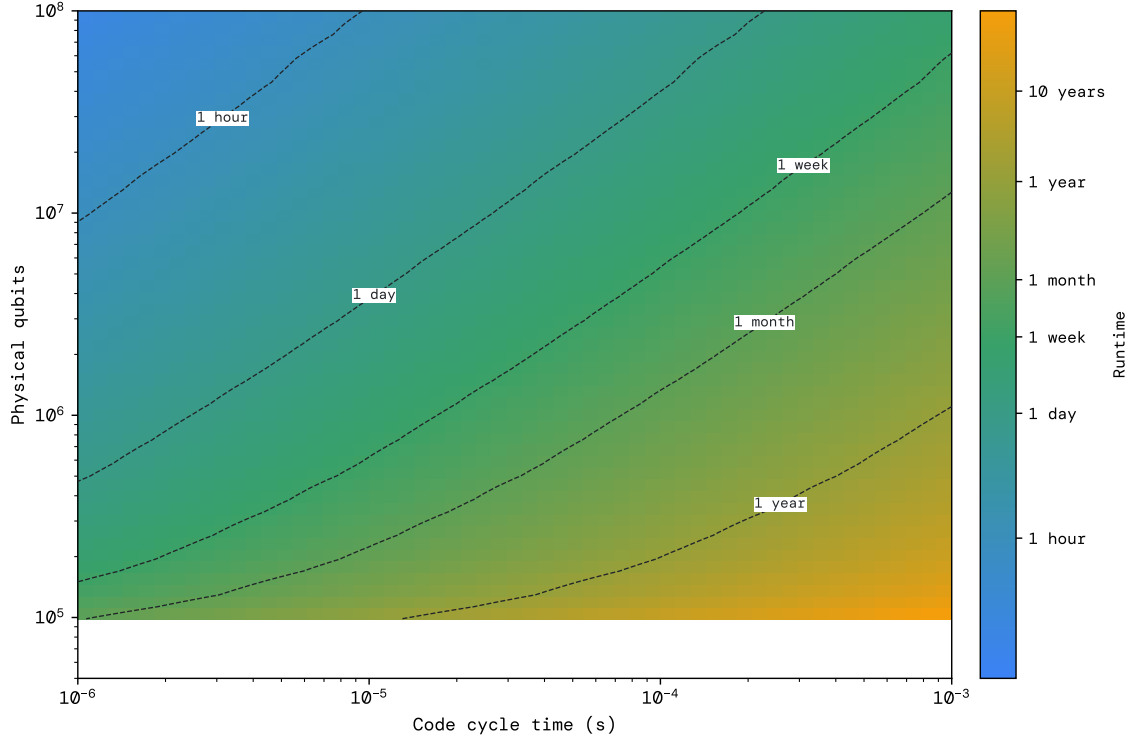
We therefore conclude that the Pinnacle Architecture can be used to achieve utility-scale quantum computation with significantly reduced overhead across multiple applications and a range of hardware regimes.

### III. BACKGROUND

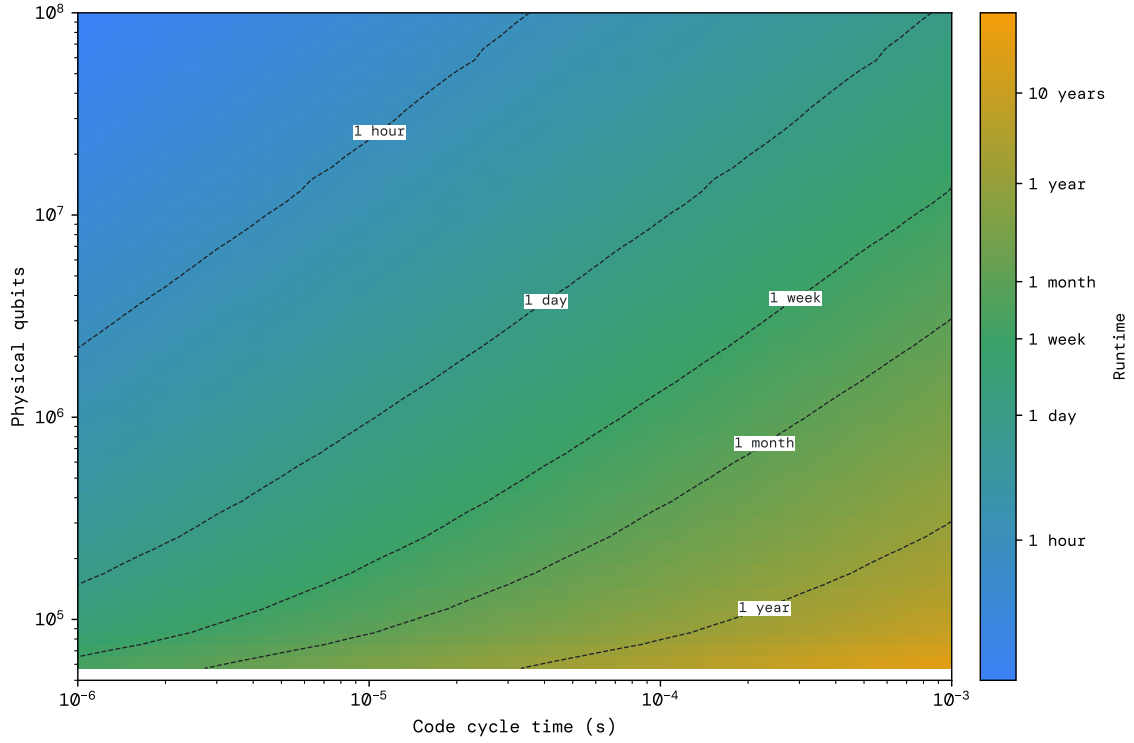
In this section we review concepts of fault-tolerant quantum computation with QLDPC codes.

#### A. Code Blocks

A code block is an instantiation of an  $[[n, k, d]]$  quantum error-correcting code. Error correction is facilitated by repeatedly performing a syndrome extraction circuit on the code block. This circuit involves measuring a set of parity check operators, which collectively yield an error syndrome. This is done with the use of  $n_c$  ancilla qubits; in each code cycle, each of these is entangled with the code qubits in accordance with one of the parity check operators and then destructively measured. A code block there-



(a)



(b)

FIG. 3. Optimal expected runtime for factoring an RSA-2048 integer on the Pinnacle Architecture as a function of the number of physical qubits and the code cycle time at physical error rates of (a)  $p = 10^{-3}$  and (b)  $p = 10^{-4}$ . White areas indicate insufficient physical qubits to implement the algorithm. The reaction time in all cases is assumed to be equal to ten times the code cycle time.

fore requires a total of  $n_{cb} = n + n_c$  physical qubits. In order to ensure robustness against measurement errors, the results of  $d_t = \Theta(d)$  code cycles must be combined to yield a reliable error syndrome; this is referred to as a *logical cycle*.

We assume the use of QLDPC codes [17, 18]. These are defined by having low parity check operator weights and qubit degrees (i.e., the number of parity checks supported on each qubit). Precisely, these check weights and qubit degrees are bounded by a constant independent of the code distance. This ensures that syndrome extraction circuits can have constant depth, and therefore be fault tolerant. The most widely used QLDPC codes are distance- $d$  (rotated) surface codes [4], which are  $[[d^2, 1, d]]$  codes with  $n_c = d^2 - 1$  parity checks which have weight at most four and require only nearest-neighbour interactions. Surface code blocks therefore use  $2d^2 - 1$  physical qubits to encode one logical qubit. However, by relaxing the requirement of nearest-neighbour interactions, more general QLDPC codes can allow for many logical qubits to be encoded in a single code block. This can allow for significant reductions in the overhead of physical qubits required per logical qubit [18]. The required non-local interactions are supported on a range of hardware platforms [19–23].

### B. Processing Blocks

To allow for fault-tolerant quantum computation, instead of only passive storage of quantum information, the concept of a code block must be generalised to a *processing block*. A processing block allows for logical operations to be implemented on its encoded logical qubits.

A QLDPC code block can be turned into a processing block by appending a measurement gadget system that allows for *generalised lattice surgery* [24–26]. This construction ensures that the combined code block-gadget system constituting the processing block remains a QLDPC code, while also ensuring that measuring a selected Pauli logical operator of the code is equivalent to the product of a set of parity check measurements on the gadget system. This allows for a logical Pauli to be measured in parallel with error correction within a logical cycle by performing a modified circuit for syndrome extraction on the full processing block [27]. Arbitrary logical Pauli measurements can then be performed across multiple processing blocks by bridging the gadget systems of these blocks [28, 29]. The number of physical qubits in the processing block is given by  $n_{pb} = n_{cb} + n_G + n_b$ , where  $n_G$  is the number of physical qubits in the gadget system and  $n_b$  is the number of physical qubits used to bridge it to another processing block.

### C. Pauli-Based Computation

When combined with injected magic states, logical measurements across bridged processing blocks support universal quantum computation on the encoded logical qubits using Pauli-based computation [13]. Indeed, a quantum circuit on  $\kappa$  qubits with a  $T$  count of  $\tau$  and any number of Clifford gates can be performed using  $\tau + \kappa$  Pauli measurements and one  $|T\rangle$  state for each of the first  $\tau$  measurements [6]. More generally, if the circuit also contains  $o$  intermediate Pauli measurements on which later operations adaptively depend, then the circuit can be performed using  $\tau + \kappa + o$  Pauli measurements without any additional  $|T\rangle$  states. This implies that such a circuit can be performed fault-tolerantly using  $\lceil \kappa/k \rceil \llbracket n, k, d \rrbracket$  bridged processing blocks in  $\tau + \kappa + o$  logical cycles.

The compilation which allows for this implementation follows that presented in Ref. [6]. First, each  $T$  gate is replaced by a magic state injection circuit, which requires one Pauli measurement with support on the processing block. Then, all Clifford gates are commuted through to the end of the circuit and absorbed into the final measurement of each of the  $\kappa$  qubits. This is done using the rule that the Pauli defining the basis of each measurement is transformed by conjugation by each Clifford that either passes through it or (in the case of the final measurements) that it absorbs. By definition, Clifford gates map Pauli operators to Pauli operators under conjugation, so the resulting circuit consists of  $\tau + \kappa + o$  Pauli measurements, along with a  $|T\rangle$  state for each  $T$  gate, as required.

### D. Relevant Timescales

There are three relevant timescales that we use in the determination of the runtime of a fault-tolerant quantum circuit. First, there is the code cycle time  $t_c$ , which is the time required for the completion of one code cycle (i.e., one round of syndrome extraction). This time is hardware-dependent, with typical estimates for different platforms ranging from 1  $\mu$ s to 1 ms [16, 22, 30–32]. Second, there is the logical cycle time  $t_l$ , which is the time required for one logical cycle. Since a logical cycle consists of  $d_t$  code cycles, this is related to the code cycle time by  $t_l = d_t t_c$ .

Finally, there is the *reaction time*  $t_r$  [7]. This is defined as the minimum time between the start of one logical measurement,  $M$  and the start of any subsequent measurement whose basis depends adaptively on the outcome of  $M$ . The reaction time is dependent on the classical control system of the quantum hardware. For simplicity, we assume throughout that the reaction time is equal to ten times the code cycle time, i.e.,  $t_r = 10t_c$ . Since  $t_c \geq 1 \mu$ s for all resource estimates we present, this implies that the reaction time always exceeds the conventionally-assumed minimum value of 10  $\mu$ s [7]. With this reaction time, our architecture is not reaction-limited provided that  $d_t \geq 10$  for all code blocks, which is true for all resource estimates



we present.

#### IV. THE PINNACLE ARCHITECTURE

In this section we present the *Pinnacle Architecture*, a low-overhead, modular and parallelisable quantum computing architecture based on QLDPC codes. We begin by presenting the modules that constitute the architecture—processing units, magic engines and memory—and then describe the overall operation, structure, and scalability of the architecture.

##### A. Modules

###### 1. Processing Units

The primary modules of the architecture are *processing units*. A processing unit uses  $\beta$  processing blocks of an  $[[n, k, d]]$  QLDPC code to allow fault-tolerant quantum computation on  $\kappa := \beta k$  logical qubits. These processing blocks can be arranged in a line with bridges connecting nearest-neighbour blocks. This allows for the measurement of an arbitrary logical Pauli operator, supported on any or all of the logical qubits in the processing unit, in each logical cycle.

###### 2. Magic Engines

To allow for universal computation, we introduce a new module, which we refer to as a *magic engine*. A magic engine allows magic states to simultaneously be produced and consumed to provide a continuous throughput of magic states to an associated processing unit.

Specifically, each processing unit is equipped with one magic engine. The magic engine produces one encoded  $|\bar{T}\rangle$  magic state in each logical cycle, and is also bridged to its associated processing unit to allow a joint measurement with that unit in parallel with state production. This is intended to ensure that in each logical cycle there is a magic state available for the processing unit to consume (i.e., the state produced in the previous logical cycle).

A magic engine can be constructed from an  $[[n_e, k_e, d_e]]$  QLDPC code block as follows. Partition the logical qubits of the code block into two halves, which we label the left ( $L$ ) and right ( $R$ ) logical sectors. In odd-numbered logical cycles, a magic state is produced by performing a magic state distillation circuit consisting of a sequence of magic state injections. These are implemented by joint logical measurements on the logical qubits of the  $L$  logical sector and small ancilla systems that hold noisy  $|T\rangle$  states. The result is that the first logical qubit of the  $L$  logical sector is in an encoded  $|\bar{T}\rangle$  state at the end of the logical cycle. In parallel, a magic state is consumed from logical

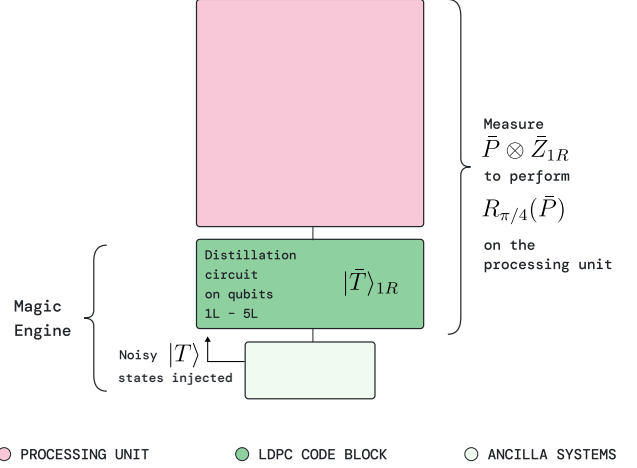


FIG. 4. Structure and operation of a magic engine. Magic state distillation is applied on one logical sector of a QLDPC code block using noisy  $|T\rangle$  states injected from ancillary systems. In parallel, an arbitrary Pauli measurement on the processing unit joint with  $\bar{Z}_1$  on the other logical sector injects an encoded  $|\bar{T}\rangle$  state that was distilled in the previous logical cycle.

sector  $R$  by performing a logical measurement of an arbitrary logical operator on the processing unit joint with  $\bar{Z}$  on the first logical qubit of  $R$ . This allows for the injection of the  $|\bar{T}\rangle$  state to perform arbitrary  $\frac{\pi}{8}$  Pauli rotations on the processing unit. To complete the injection, an  $\bar{X}$  measurement is also required on the first logical qubit of  $R$ ; a change of basis can be performed between logical cycles to allow that logical qubit to be offline while it is performed in parallel with the next logical cycle. In even-numbered logical cycles, the roles of the two logical sectors are swapped.

Since magic state distillation relies on post-selection, for each logical cycle there is some probability  $p_r$  that the state produced by the magic engine is rejected. When this happens—and if a magic state is required for the next logical cycle—the processing unit can be left idle for the next logical cycle to allow for a new magic state to be prepared. This causes the expected number of logical cycles required per  $T$  gate to increase from 1 to  $\alpha = (1 - p_r)^{-1}$ . The distillation protocol should be chosen such that  $p_r$  is small to ensure that this effect is also small.

###### 3. Memory

The architecture can also include memory. This is optional, but it is useful in cases where a large number of logical qubits must be stored but not processed. It consists of  $\nu$  code blocks of an  $[[n_m, k_m, d_m]]$  quantum error-correcting code encoding  $\mu = \nu k_m$  logical qubits. Since these logical qubits are not processed, full processing blocks are not required. However, to facilitate rearrangement of the memory, we ensure that each code block has  $n_m$  ancilla qubits (including those used for syndrome

extraction). This means that the memory consists of a total of  $2\nu n_m$  physical qubits.

Memory is accessed by processing units via *ports*. To facilitate this, we partition the logical qubits of the memory into sets of size  $w$ , which we refer to as *windows*. For simplicity, we enforce the condition that each window is contained within a single code block; this implies that  $w$  divides  $k_m$  such that there are a total of  $\nu k_m/w$  windows. Each port is associated with one window and consists of a gadget that allows for arbitrary logical  $Z$ -type measurements on the  $w$  logical qubits of that window. Bridging a processing unit to a port then allows for any circuit with arbitrary gates on the processing unit and controls on the  $w$  logical qubits of the memory window to be implemented (via Pauli-based computation). This is sufficient to allow read-only access to that window of memory by the processing unit [33]. More generally, we can assign a port to any subset of the windows of the memory. Provided the gadgets constituting each of these ports allow for arbitrary logical  $Z$  measurements to be performed in parallel, this can allow for up to  $\nu k_m/w$  processing units to access the memory in parallel.

For each processing unit to access the full memory, the memory code blocks must be permuted. To ensure that this does not require arbitrary routing, we impose the constraint that each code block is to be shifted by at most one position per logical cycle. We may then perform the required permutation without requiring connectivity on a longer scale than the size of a code block as follows. The  $\nu$  memory code blocks are arranged such that the  $i$ th and  $(i+1)$ th (mod  $\nu$ ) code blocks are adjacent for  $1 \leq i \leq \nu$  (e.g., in a loop). A cyclic shift of memory code blocks is performed by physical SWAPs of the  $j$ th data qubit in code block  $i$  with the  $j$ th ancilla qubit in code block  $i+1$  (mod  $\nu$ ) for  $1 \leq i \leq \nu$  and  $1 \leq j \leq n$ , followed by a (local) SWAP of the  $j$ th data qubit and  $j$ th ancilla qubit in each code block. Since such a circuit requires only one layer of non-local gates confined to the scale of a code block, whereas a QLDPC syndrome extraction circuit generally requires many such gates, we assume that this rearrangement can be completed during a code cycle and so its time cost is negligible. By applying  $\nu$  of these cyclic shifts over a period of at least  $\nu$  logical cycles, every window of memory can be accessed by each processing unit.

## B. Operation

We now consider how the architecture operates while performing a quantum computation. To aid understanding, we start with a simplified baseline operation (Section IV B 1) and incrementally build up to the most general operation (Section IV B 4).

### 1. Serial Operation

As a baseline, let us first consider a serial mode of operation. In this mode, there is a single processing unit with  $\kappa$  logical qubits (and, for simplicity, we assume there is no memory). During each logical cycle, a joint logical Pauli measurement is performed on the processing unit and magic engine. In parallel, the magic engine produces a magic state for the next logical cycle. Accounting for a magic engine reject rate of  $p_r$ , this allows for an arbitrary Clifford+T circuit on  $\kappa$  qubits with a  $T$  count of  $\tau$  and  $o$  intermediate measurements to be performed fault-tolerantly in an average of  $\tau/(1-p_r) + \kappa + o$  logical cycles.

### 2. Fully Parallel Operation

As a next step, we can consider the case of implementing a circuit that can be completely separated out into two or more independent circuits. In this context, we can separate the architecture up into a separate processing unit for each independent circuit and perform all the circuits in parallel. This reduces the number of logical timesteps required from  $\tau/(1-p_r) + \kappa + o$  to approximately  $\max_i (\tau_i/(1-p_r) + \kappa_i + o_i)$  where  $\tau_i$ ,  $\kappa_i$  and  $o_i$  denote the number of  $T$  gates, logical qubits and intermediate logical measurements in the  $i$ th independent circuit. This expression omits an  $O(\sqrt{\max_i (\tau_i)})$  correction arising from variance in the proportion of magic states rejected in different processing units across the duration of the circuit. Since we are interested in circuits where the  $T$  count is large, we assume this correction is negligible.

An example of where this mode could be used is in implementing multiple shots of an algorithm in parallel. In this context, it can be considered a way to use a greater number of qubits to reduce the runtime compared to when all shots are performed in series.

### 3. Flexibly Parallel Operation

More common and general is the case where a circuit can be implemented partially in parallel. In such a circuit, no subset of logical qubits is entirely separable from the rest, but significant parts of the circuit involve operations on disjoint registers of logical qubits. A conventional circuit implementation would allow such parts of the circuit to be performed in parallel on the disjoint registers. We now show how this can be done in the Pinnacle Architecture.

Parallelism of this kind is inherently challenging with Pauli-based computation because of the effect of commuting Clifford gates through to the end. To see this, consider the case of a *Clifford frame* at a given point in the circuit

(i.e., the product of Clifford gates up to that point) that corresponds to an entangling gate between two processing units. Then, as Cliffords are commuted through the circuit the supports of the logical measurements on one processing unit will spread out to straddle both. In particular, this means that a logical measurement corresponding to a  $|\bar{T}\rangle$  state injection—required to perform a  $\bar{T}$  gate on either processing unit—comes to have support on both units. Since each processing unit only allows one logical measurement per logical cycle, this implies that a  $\bar{T}$  gate on a logical qubit on one processing unit cannot be performed in parallel with a  $\bar{T}$  gate acting on a logical qubit of the other processing unit.

Parallelism therefore requires that the Clifford frame acts as a tensor product across the units which are to be parallelised. One way to achieve this (following Ref. [34]) could be to perform  $\overline{\text{CNOT}}$  gates that entangle processing units physically using additional logical measurements, so that they can be excluded from the Clifford frame. However, this approach leads to a time cost that scales with the number of  $\overline{\text{CNOT}}$  gates, which can quickly cause the benefits of parallelism to be erased. In particular, it performs poorly in the common setting where one part of the circuit is highly parallelisable but another is not, since the cost of parallelising the former part scales with the number of entangling gates in the latter part.

We instead propose a more flexible alternative that allows for parallelism when it is beneficial but avoids the cost of physically implementing every  $\overline{\text{CNOT}}$  gate. To develop this approach, we introduce the technique of *Clifford frame cleaning*. Precisely, let  $\mathbf{K}$  be a set of logical qubits and  $\mathbf{K}' \subset \mathbf{K}$  be a subset of  $|\mathbf{K}'|$  of these logical qubits. If  $C$  is a Clifford frame acting on  $\mathbf{K}$ , cleaning  $C$  off  $\mathbf{K}'$  means physically performing a Clifford  $U$  such that  $CU$  acts trivially on  $\mathbf{K}'$ . We show in Lemma 1 that this can be done using at most  $4|\mathbf{K}'|$  logical Pauli product measurements on  $\mathbf{K}$ .

Harnessing this tool, our flexible parallelism framework is as follows. We consider processing units to automatically be *joined* into larger units at any point in the circuit where there is an entangling gate between the units. From that point on,  $\bar{T}$  gates and logical measurements on any of the constituent processing units of this joined unit are assumed to require joint logical Pauli product measurements across the entire joined unit, meaning that only one such gate can be implemented on the joined unit per logical timestep. At any later time, we can then *separate* a processing unit (with  $\kappa$  logical qubits) from a joined unit by cleaning the Clifford frame off the joined unit, at a cost of at most  $4\kappa$  additional logical timesteps. From then on, logical measurements on the separated unit can again be performed in parallel with the unit it was separated from. This process is shown in Fig. 5.

This framework allows for processing units to be joined during parts of a circuit in which many inter-unit gates occur, but then to be separated again (at a relatively small cost) for parts of the circuit that are more amenable to par-

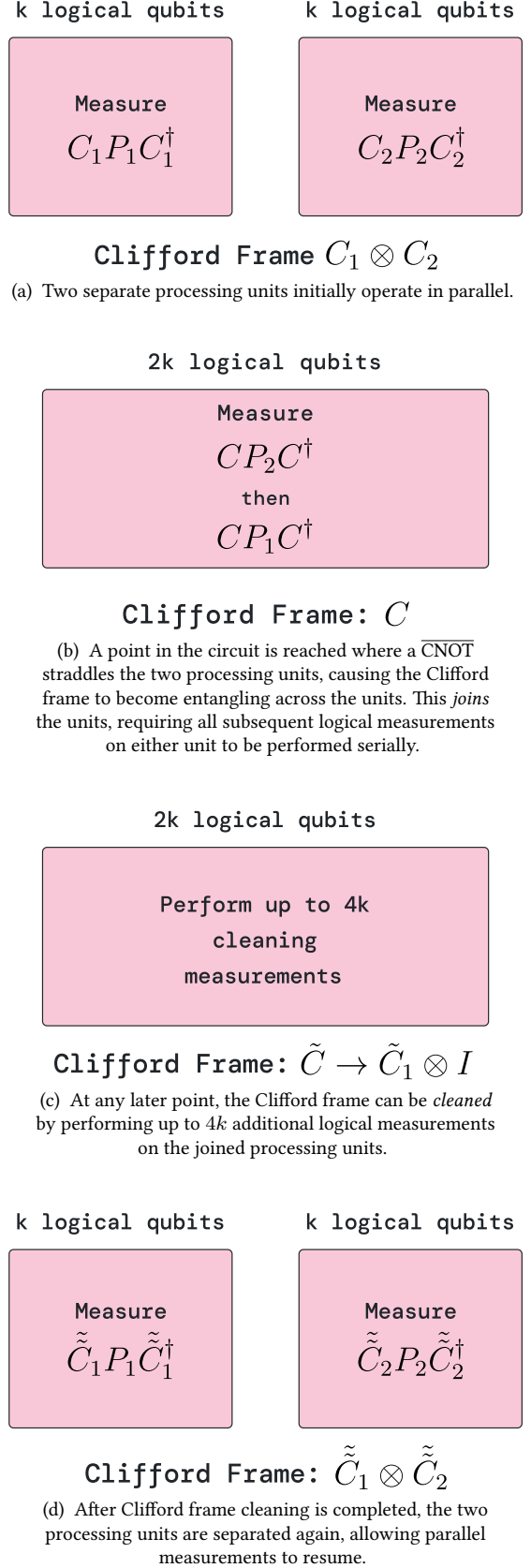


FIG. 5. Process of joining and separating processing units with Clifford frame cleaning to allow for flexible parallelism.



allelisation. The choice of if and when to separate units can be specifically optimised for compilation of any particular circuit, with the potential for significant time savings compared to either a fully serial approach or an approach that depends on physical implementation of all inter-unit entangling gates.

#### 4. General Operation

The final step to our fully general operation is to optionally incorporate the memory. Recall that each processing unit accesses memory via a port. We allow for read-only memory access, which requires only gates that act as a control on the port and a target on the processing unit [8, 33]. This means that the access can be provided by using logical CNOT gates with controls on the logical qubits in a port and targets on ancillary logical qubits in the processing unit to fan out memory data onto the processing unit at the start of the access and fan in at the end of the access. Since such operations commute, arbitrarily many processing units can access the memory in parallel, provided they each have ports with measurement gadgets that can be used in parallel.

Implementing memory access on the Pinnacle Architecture uses the same concepts of joining and separating units presented in Section IV B 3. Specifically, when a processing unit accesses a window of the memory, the port associated with that window is joined onto the processing unit by the logical CNOTs which implement the fan-out. Since no entangling gates act within the memory, these ports can always be assumed to be separate from one another. Moreover, as all gates between a port and processing unit act as controls on the port, commuting through the Clifford frame only gives rise to  $Z$ -type logical measurements on the port. When the access is finished, the Clifford frame is cleaned off the port such that the port can be separated from the processing unit again. As shown in Lemma 2, this requires only  $2w$  logical cycles for a port of  $w$  qubits. This ensures that subsequent logical measurements on the processing unit act trivially on the memory logical qubits, enabling subsequent access by different processing units.

#### C. Scalability

The architecture is designed to ensure that its operation remains feasible at large scale. Specifically, because the processing units are assembled from individual processing blocks that are all connected via bridges between nearest neighbours, logical operators with support across any subset of processing blocks in a processing unit can be measured using connections between physical qubits that are restricted to the scale of one processing block. This means that arbitrarily large processing units supporting arbitrarily many logical qubits can be realised with phys-

ical connections of constant scale. For example, in a two-dimensional arrangement of qubits, this scale is approximately the square-root of the size of the processing block  $\sqrt{n_{pb}}$ .

This means that the architecture can be supported even on hardware platforms that only support interactions whose fidelity decreases continuously with the interaction distance. Moreover, it means that no routing of logical qubits across the architecture is required. Instead, all changes between logical cycles required to support different logical operations are confined to the scale of a processing block. As discussed in Section V A, code choice and gadget design can minimise the dynamism required even on this scale to be very limited. In summary, the significant overhead reductions supported by QLDPC codes can be realised with interactions and rewiring confined to a fixed scale, which can be chosen to be consistent with a given hardware platform.

The further modularisation of the architecture into processing units provides an additional benefit. Specifically, feasibility limitations are expected to require large-scale quantum computers on many hardware platforms to be assembled from smaller modules [9, 10, 35]. This opens up an opportunity for hardware and architecture co-design that can be realised by associating these hardware modules with processing units of the Pinnacle Architecture. This is beneficial because it aligns compilation imperatives with hardware constraints. Gates between hardware modules should be minimised since they are likely to have poorer performance than intra-module gates, while gates between processing units should also be minimised to maximise parallelism.

#### D. Structure

The only constraint on the assembly of the architecture is that modules that are joined together at any time must be connected in the architecture. This ensures that these modules can be bridged, and therefore logical measurements are possible across them, without requiring long-distance transport. In particular, this means that each processing unit must be adjacent to its associated magic engine and—if a memory is present—a port of memory. If a set of processing units is to be joined, these processing units should also be adjacent to each other.

The structure of the architecture for different example instantiations is shown in Fig. 1.

### V. INSTANTIATION OF THE ARCHITECTURE

In this section, we present a specific instantiation of the Pinnacle Architecture using a family of generalised bicycle (GB) codes, along with numerical simulation results used to determine the logical error rates that can be achieved for different of code distances and physical error

rates.

### A. Setup

GB codes are defined by a lift  $l \in \mathbb{N}$  and sets  $A, B \subseteq \mathbb{Z}_l$  [36]. They have  $n = 2l$  physical qubits which can be divided into two sectors of  $l$  physical qubits each, which we label  $L$  and  $R$ . The parity check operators of the code are then

$$S_{X,j} = \prod_{a \in A} X_{(j+a),L} \prod_{b \in B} X_{(j+b),R}, \quad (1)$$

$$S_{Z,j} = \prod_{a \in A} Z_{(j-a),R} \prod_{b \in B} Z_{(j-b),L}. \quad (2)$$

Here, the first subscript on each operator denotes the position of a qubit within the sector and the second denotes the sector. For any  $\sigma \in \mathbb{Z}_l$ , a cyclic shift of all physical qubits by  $\sigma$  sites preserves the group of parity check operators, making it a qubit automorphism.

We construct code blocks from the specific family of GB codes presented in Ref. [11], first explored in Ref. [37]. These codes have weight-six parity check operators and require only simple, relatively short-distance transport patterns for syndrome extraction. The code family is parameterised by an integer  $m > 3$ , and defined by choosing the lift to be  $l = 2^m - 1$ , as well as the sets  $A$  and  $B$  such that the polynomials  $A(x) = \sum_{a \in A} x^a$  and  $B(x) = \sum_{b \in B} x^b$  generate the parity check matrices of the classical simplex codes. Classical simplex codes have parameters  $[2^m - 1, m, 2^{m-1}]$  [38]. We conjecture that the GB codes constructed in this way have parameters  $[[2(2^m - 1), 2m, m + (m - 4)^2]]$ . We present explicit instances for the first five codes in the family in Table I.

For these codes, we empirically find that performance is improved by allowing for slightly more rounds of syndrome extraction than the code distance. Guided by this observation, we choose to use  $d_t = d + 2$  code cycles per logical cycle.

The code blocks are extended to processing blocks by supplementing them with the gadget system presented in Ref. [11]. This gadget system consists of four gadgets which correspond to four seed operators chosen such that all logical Pauli operators are products of cyclic shifts of the seed operators. In particular, the  $k$  logical qubits of each code naturally divide into two logical sectors ( $L$  and  $R$ ) with  $k/2$  logical qubits in each such that one  $X$ -type and one  $Z$ -type seed operator suffice for each logical sector. Constructing, bridging, and shifting four gadgets capable of measuring each of the seed operators therefore suffices to measure arbitrary logical Pauli operators on a code block. This means that only minor alterations are required to allow the same syndrome extraction circuit to measure any logical Pauli operator. Four bridges are also included per processing block; three are used to bridge the four gadgets within the block, while the fourth is used to

bridge the last gadget of the block to the first gadget of the next block.

Therefore, letting  $n_g$  be the number of physical qubits per gadget and  $n_b$  be the number of physical qubits per bridge, and accounting for  $n_c = n$  check qubits, the total number of physical qubits per processing block is given by:

$$n_{pb} = n_{cb} + 4n_g + 4n_b. \quad (3)$$

The values of these parameters are provided in Table I.

These gadgets can also be used to measure certain sets of logical operators in parallel (i.e., in a single logical cycle) via the inclusion of duplicate gadgets. Specifically, a set of  $m$  logical operators  $P_1, \dots, P_m$  can be measured in parallel by using a different copy of the gadget to measure each, provided they commute on every physical qubit. This qubit-wise commutation condition ensures that the check operators from different gadgets all commute. Connecting  $m$  gadgets to the same code block can increase code check weights and qubit degrees in places where multiple gadgets are joined to the same qubit or check. In theory, this increase can be by up to  $m$  but, with in an appropriately chosen basis, it is typically significantly less than this. If necessary, a small number of ancillary qubits can also be used to reduce check weights and/or qubit degrees [25, 39].

### B. Modules

#### 1. Processing Units

Using  $\beta$  processing blocks constructed from the GB code family introduced above (for any  $\beta \in \mathbb{N}$ ), we can encode  $\kappa = \beta k$  logical qubits in  $\beta n_{pb}$  physical qubits. Specifically, with a code distance of  $d = 16$ , we can encode  $14\beta$  logical qubits in  $860\beta$  physical qubits. For better protection, we can instead use a code distance of  $d = 24$  and encode  $16\beta$  logical qubits in  $1620\beta$  physical qubits.

#### 2. Magic Engines

We construct each magic engine from code blocks of the same GB code family as those used for the processing blocks. These blocks naturally have the required  $L$  and  $R$  logical sectors, with  $k/2 > 5$  logical qubits in each sector when  $d \geq 10$ .

We use 15-to-1 magic state distillation on these code blocks to produce encoded  $|\bar{T}\rangle$  magic states [40]. Following Ref. [41], this can be done using fifteen auto-corrected  $Z$ -type  $\pi/8$  rotations, followed by four logical  $X$ -type measurements used for post-selection. Each of these rotations is implemented by injecting a noisy  $|T\rangle$  from a bridged pair of small ancillary  $[[n_a, 1, d_a]]$  codes. The  $|\bar{T}\rangle$  state is prepared in the first of these codes using standard techniques (e.g., using a post-selected state injection

TABLE I. Instances of the family of generalised bicycle codes. For each code, the parameters  $\llbracket n, k, d \rrbracket$  are provided, along with the number of code cycles per logical cycle,  $d_t = d + 2$ . The codes are defined by  $l$ ,  $A$ , and  $B$ , alongside Eq. (1) and Eq. (2). The remaining columns show the number of physical qubits in their code blocks, gadgets, bridges and processing blocks. See Ref. [11] for more details.

| $\llbracket n, k, d \rrbracket$     | $d_t$<br>( $d + 2$ ) | $l$ | $A$              | $B$              | Code Block Qubits<br>( $n_{cb} = 2n$ ) | Gadget Qubits<br>( $n_g$ ) | Bridge Qubits<br>( $n_b$ ) | Processing Block Qubits<br>( $n_{pb} = n_{cb} + 4n_g + 4n_b$ ) |
|-------------------------------------|----------------------|-----|------------------|------------------|----------------------------------------|----------------------------|----------------------------|----------------------------------------------------------------|
| $\llbracket 30, 8, 4 \rrbracket$    | 6                    | 15  | $\{0, 6, 13\}$   | $\{0, 1, 4\}$    | 60                                     | 13                         | 7                          | 140                                                            |
| $\llbracket 62, 10, 6 \rrbracket$   | 8                    | 31  | $\{0, 6, 15\}$   | $\{0, 5, 7\}$    | 124                                    | 19                         | 11                         | 244                                                            |
| $\llbracket 126, 12, 10 \rrbracket$ | 12                   | 63  | $\{0, 4, 37\}$   | $\{0, 29, 49\}$  | 252                                    | 31                         | 19                         | 452                                                            |
| $\llbracket 254, 14, 16 \rrbracket$ | 18                   | 127 | $\{0, 32, 100\}$ | $\{0, 28, 49\}$  | 508                                    | 57                         | 31                         | 860                                                            |
| $\llbracket 510, 16, 24 \rrbracket$ | 26                   | 255 | $\{0, 39, 55\}$  | $\{0, 70, 127\}$ | 1020                                   | 99                         | 51                         | 1620                                                           |

circuit [42] or, if necessary to increase the input state fidelity, zero-level distillation [43, 44] or magic state cultivation [45, 46]). The second of the codes provides an ancillary logical qubit for auto-correction, to account for Clifford corrections from the state injection. Parallel joint logical measurements (repeated  $d_a$  times) between the  $L$  logical sector of the GB code and the first ancillary code, and between the two ancillary codes, are performed by generalised surgery, followed by destructive measurements of the ancillary codes, to implement the auto-corrected injection circuit presented in Fig. 13c of Ref. [41]. Parallel  $X$ -type measurements (repeated  $r$  times) are then performed on qubits 2–5 of the  $L$  logical sector and the state rejected if any of these four measurement results is  $-1$ .

The physical qubits that must be accounted for in this protocol are the GB code block itself ( $n_{cb}$ ), sixteen gadgets used to perform up to sixteen logical measurements in parallel ( $16n_g$ ), fifteen pairs of ancillary  $\llbracket n_a, 1, d_a \rrbracket$  codes from which states are injected ( $2 \times 15(2n_a - 1)$ ), fifteen pairs of bridges used to connect these ancillary codes to each other and to the GB code block and (if necessary) ancillary qubits ( $n_\alpha$ ) used to reduce the infidelity of input states (e.g., by magic state cultivation). The total overhead of the magic engine is therefore given by

$$n_{me} = n_{cb} + 16n_g + 60(n_a + d_a - 1) + n_\alpha. \quad (4)$$

The protocol produces output  $|T\rangle$  states of fidelity  $p_T$ , which can be arbitrarily close to  $35p_{in}^3$  with a rejection rate of approximately  $15p_{in}$ , where  $p_{in}$  is the infidelity of the input noisy states [41]. Specifically, to achieve this we require that the distance of the QLDPC code block,  $d_e$ , is large enough that the logical error rate of the QLDPC code block is  $\ll p_T$ . Hence, the contribution to the output fidelity from logical errors in the QLDPC code block is negligible. We also require that  $r$  is large enough that the logical error rate of each  $X$ -type logical measurement is  $\ll p_T/p_{in}$ , so that the probability of an error in a single  $Z$  rotation both occurring and failing to be detected is negligible. Finally, we require that  $d_a$  is large enough that the logical error rate of each ancillary code is  $\ll p_{in}$ , so that the contribution to the probability of errors in each  $Z$  rotation from logical errors during state injection or auto-correction is negligible. The protocol completes within the required time of one logical cycle ( $t_l = d_t t_c$ ) provided

that  $d_a + r \leq d_t$ .

In practice, for the applications in Section VI, an output fidelity of  $p_T \approx 10^{-11}$  is sufficient, which is achievable provided with an input magic state infidelity of  $p_{in} \approx 10^{-4}$ .

For a physical error rate of  $p = 10^{-4}$ , the desired output fidelity can be achieved by a state injection circuit into a colour code with post-selection [42]. Since this protocol takes only  $d_a + 1$  colour code cycles for a distance- $d_a$  colour code, it can be attempted multiple times for each colour code, which—given the post-selection acceptance rate of 2%—makes the probability of failure to produce the required states in each logical cycle negligible. This means that we can assume the overall rejection rate of the magic engine to be  $p_r = 15p = 0.15\%$ . A distance of  $d_a = 5$  for the pair of ancilla codes suffices to achieve the requirement of a logical error rate much smaller than  $p_{in}$  [47]. Using the results of Section VC, GB code blocks of distance  $d_e = 10$  and  $r \geq 7$  rounds of  $X$ -type measurements also ensure a sufficiently low logical error rate from the QLDPC code and completion within one logical cycle for processing block codes of  $d \geq 10$  [48]. With these considerations, the total physical qubit count for the magic engine in this case is  $n_{me,10^{-4}} = 2128$ .

For higher physical error rates, the input magic state can be prepared in a rotated surface code using fold-transversal cultivation [46]. In particular, we consider the case of  $p = 10^{-3}$  and  $d = 24$  processing blocks. In this case,  $d_a = 9$  rotated surface codes are sufficient for a logical error rate much smaller than  $p_{in}$  [49]. Fold-transversal cultivation into such codes can be performed to prepare states with an infidelity of  $p = 10^{-4}$  with a success rate of approximately  $2/3$ . Post-selection on the final surface code requires only three code cycles [46] and there are  $d_t - d_a = 17$  code cycles during which injection into the GB code is not taking place. Hence, the preparation can be attempted five times, meaning that at least one success on each of the fifteen required surface codes with a probability of  $(1 - 1/3^5)^{15} = 94\%$ . By comparison, the magic state distillation reject rate of  $15p_{in} = 0.15\%$  is negligible, so we can assume that the overall reject rate is  $p_r = 6\%$ . To provide a sufficient throughput of cultivated states that can be escaped into these codes, two copies of a 25 physical qubit ancilla system are required per sur-

face code [46], leading to an additional ancilla overhead of  $n_\alpha = 2 \times 15 \times 25 = 750$ . As shown in Section VC, GB code blocks of distance  $d_e = 24$  also satisfy the requirement of a logical error rate much smaller than  $p_T$ . We also find that  $r \geq 13$  rounds of logical  $X$  type measurements suffice to achieve a logical error rate from these measurements of  $\ll p_T/p_{\text{in}} = 4 \times 10^{-7}$ . This ensures that the protocol completes within a  $d = 24$  GB logical cycle of  $d_t = 26$  code cycles. With these considerations, the total physical qubit count for the magic engine in this case is  $n_{me,10^{-3}} = 8694$ .

### 3. Memory

For the memory, we use the same code blocks as are used for the processing blocks. For simplicity, we match the window size with the number of logical qubits in a logical sector,  $k/2$ . Each port then corresponds to one of the  $Z$ -type gadgets used in the gadget system of the processing blocks, along with a bridge to connect to a processing unit. The logical operators that must be measured to access the memory commute on every physical qubit and so can be measured in parallel, as they act across disjoint processing units and act only as  $Z$ -type operators on the memory. Moreover, since each memory code block has at most two ports, these measurements increase the check weight and qubit degree by at most two.

Referring to Table I, the additional number of physical qubits per port is  $n_g + n_b = 88$  at  $d = 16$  or  $n_g + n_b = 150$  at  $d = 24$ . Hence, for any  $\nu \in \mathbb{N}$ , we can encode  $14\nu$  logical qubits in memory such that  $\rho$  processing units can access it in parallel with  $508\nu + 88\rho$  physical qubits at  $d = 16$  or  $16\nu$  logical qubits in memory with  $1020\nu + 150\rho$  physical qubits at  $d = 24$ .

### C. Simulation Results

To assess the logical error rates achievable with different code choices, we perform numerical simulations of both memory and logical measurements by generalised surgery. These are performed for one logical cycle ( $d_t = d + 2$  rounds of syndrome extraction) with standard circuit-level depolarising noise. For memory experiments we instead simulated  $d$  rounds of syndrome extraction and rescaled the logical failure rates by  $(d + 2)/d$ . Circuits for the memory simulations are constructed as in Ref. [37], while circuits for the surgery simulations are constructed using integer linear programming [50]. We perform uncorrelated (only  $X$ -type detectors) most-likely error decoding by converting the decoding problem into a mixed integer program and allow the solver to obtain an optimal solution. Results of these simulations are shown in Fig. 6.

We assume each point is binomially distributed and use maximum likelihood estimation to fit the points to the

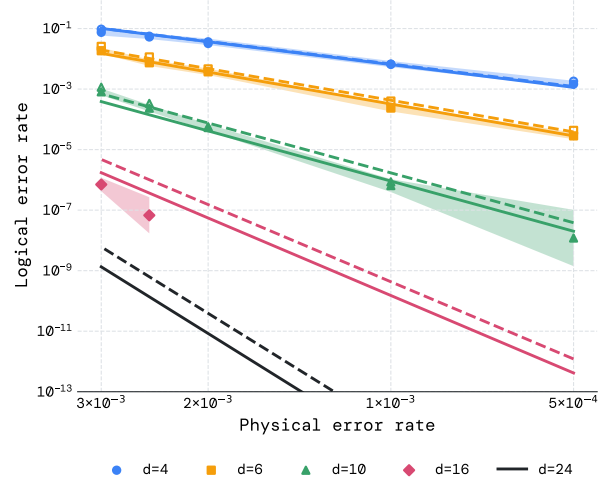


FIG. 6. Simulation results to determine logical error rates per processing block (of  $k$  logical qubits) per logical cycle of the Pinnacle Architecture using GB codes of different distances and across different physical error rates. Solid markers show results for  $X$ -basis memory experiments decoded with most-likely error decoding. Hollow markers show results for logical measurements by generalised surgery; data was collected for the  $d = 4$ ,  $d = 6$  and  $d = 10$  codes at the same physical error rates as for the memory experiments. Highlighted regions are 99% confidence intervals for the memory experiment data points. The solid and dashed lines are fits of the ansatz in Eq. (5) to the memory experiment data and logical measurement data respectively. We emphasise that ansatz parameters are independent of distance (i.e., the same equations are used for all codes), which allows for extrapolation of the collected data across the code family.

sub-threshold ansatz

$$p_{L,cb}(p, d) = A \left( \frac{p}{B} \right)^{\frac{d}{2} + C}. \quad (5)$$

Here  $p_L$  is the total logical failure rate for all  $k$  logical observables over  $d_t$  rounds. It follows from this that the error rate per logical observable and logical cycle is given by

$$p_L(p, k, d) = \frac{A}{k} \left( \frac{p}{B} \right)^{\frac{d}{2} + C}. \quad (6)$$

The fitted parameters for the ansatz with 95% confidence intervals are given in Table II. Table III shows the logical error rates per logical qubit per logical cycle derived from this ansatz for each of the GB codes considered at physical error rates of  $p = 10^{-3}$  and  $p = 10^{-4}$ .

We emphasise that the purpose of these simulation results is to benchmark the capabilities of the architecture and guide code distance choice for resource estimation. This informs the decision to use most-likely error decoding which correctly decodes all faults of weight less than  $\frac{d}{2}$  and avoids error floors that can arise from alternatives



TABLE II. Fitted parameters with 95% confidence intervals for the ansatz in Eq. 5 for memory experiments and logical measurement experiments.

| Experiment | A                   | B                            | C                      |
|------------|---------------------|------------------------------|------------------------|
| Memory     | $5.9^{+1.8}_{-1.4}$ | $0.0179^{+0.0006}_{-0.0006}$ | $0.50^{+0.09}_{-0.09}$ |
| Log. Meas. | $6.2^{+1.9}_{-1.4}$ | $0.0158^{+0.0007}_{-0.0007}$ | $0.47^{+0.09}_{-0.09}$ |

TABLE III. Error rates per logical qubit and logical cycle for logical measurement in GB codes of each distance at physical error rates of  $p = 10^{-3}$  and  $p = 10^{-4}$ . Values correspond to the central estimates of the fit parameters from the Logical Measurement row of Table II substituted into the ansatz presented in Eq. (6).

| $p$       | $d = 4$            | $d = 6$            | $d = 10$            | $d = 16$            | $d = 24$            |
|-----------|--------------------|--------------------|---------------------|---------------------|---------------------|
| $10^{-3}$ | $8 \times 10^{-4}$ | $4 \times 10^{-5}$ | $1 \times 10^{-7}$  | $3 \times 10^{-11}$ | $4 \times 10^{-16}$ |
| $10^{-4}$ | $3 \times 10^{-6}$ | $1 \times 10^{-8}$ | $5 \times 10^{-13}$ | $1 \times 10^{-19}$ | $1 \times 10^{-28}$ |

such as belief propagation decoders [51]. The problem of developing a sufficiently fast decoder for real-time use by the classical control system of quantum hardware is outside the scope of this paper, and we look forward to addressing it in future work.

## VI. APPLICATIONS

In this section, we show how the Pinnacle Architecture can be applied to two applications: determining the ground state energy of the Fermi-Hubbard model, and factoring RSA integers.

### A. Fermi-Hubbard Model

In this subsection, we determine the resources required to estimate the ground state energy of the two-dimensional Fermi-Hubbard model using the Pinnacle Architecture.

#### 1. Algorithm

The two-dimensional Fermi-Hubbard model represents a system of interacting fermions and has the Hamiltonian

$$H = H_h + H_I$$

$$= \sum_{\langle i,j \rangle} \sum_{\sigma \in \{\uparrow, \downarrow\}} \left( a_{i,\sigma}^\dagger a_{j,\sigma} + a_{j,\sigma}^\dagger a_{i,\sigma} \right) + u \sum_i \hat{n}_{i,\uparrow} \hat{n}_{i,\downarrow}. \quad (7)$$

Here,  $i$  denotes the sites of an  $L \times L$  lattice,  $\langle i, j \rangle$  denotes pairs of nearest neighbours on this lattice,  $\sigma \in \{\uparrow, \downarrow\}$  denotes spins states,  $a^\dagger$  and  $a$  represent creation and annihilation operators, respectively,  $\hat{n} = a^\dagger a$  denotes the num-

ber operator, and  $u$  denotes the (dimensionless) coupling strength.

We follow Ref. [14] in using plaquette Trotterisation to determine the ground state energy of the Fermi-Hubbard model with a relative error of 0.5% of the total lattice energy. In order to minimise the number of qubits required, we modify this method by omitting Hamming weight phasing (which requires a system of  $1 \leq \alpha \leq L^2/2$  ancillary logical qubits), instead simply performing each required phase rotation in sequence. With this simplification, the number of logical qubits required is

$$N = 2L^2 + 2. \quad (8)$$

This accounts for two logical qubits for each lattice site (one for each spin state) and two additional logical ancilla qubits (one for phase estimation and one for repeat until success synthesis) [14]. From Eq. (F10) of Ref. [14], the  $T$  count for one shot of the algorithm is given by

$$\tau = 6.203 \sqrt{\frac{W}{(\varepsilon(1-x))^3}} \times \left( N_R \left( 1.15 \log_2 \left( \frac{N_R \sqrt{3W}}{x \sqrt{1-x} \sqrt{\varepsilon^3}} \right) + 9.2 \right) + N_T \right). \quad (9)$$

Here,  $N_T = 12L^2$  and  $N_R = 4L^2$  are the number of  $T$  gates and arbitrary  $Z$  rotations (which are each synthesised from a number of  $\bar{T}$  gates) required per Trotter step. The number of Trotter steps required for the target precision is given by the prefactor. In this prefactor,  $\varepsilon = 0.005E_0$  corresponds to the required relative error of 0.5%, where the energy per site  $E_0$  is estimated at 1.02 hartrees for  $u = 4$  and 0.74 hartrees for  $u = 8$  [12].  $W$  is a parameter that bounds the Trotter error; bounds on this parameter are provided in Ref. [14]. The parameter  $x$  reflects a choice of how to split the error budget across different sources; we optimise over this in our analysis.

In addition to  $T$  gates, the algorithm also requires additional logical measurements. In particular, repeat-until-success synthesis is used to perform each of the  $N_R$  arbitrary  $Z$  rotations in each Trotter step [52, 53]. This method successfully performs the correct rotation with a probability of at least one half [53], and so the expected number of attempts per rotation is at most two. Since one logical measurement is required per attempt (in addition to the  $T$  gates), this adds an average of up to two logical measurements per arbitrary rotation. Logical measurements are also required for the final phase estimation. However, since this is only performed once (not for each Trotter step), it is negligible compared with the  $T$  count. We therefore express  $\mathcal{T}$ —the sum of the  $T$  count and number



of logical measurements—as

$$\mathcal{T} = 6.203 \sqrt{\frac{W}{(\varepsilon(1-x))^3}} \times \left( N_R \left( 1.15 \log_2 \left( \frac{N_R \sqrt{3W}}{x\sqrt{1-x}\sqrt{\varepsilon^3}} \right) + 11.2 \right) + N_T \right). \quad (10)$$

This quantity determines the number of logical cycles on the Pinnacle Architecture. It is approximately constant in  $L$  because the allowed error is relative to the total energy, which scales as  $L^2$  [12].

## 2. Implementation and Results

Concretely, we consider the case of even  $L \leq 32$  and  $u = 4$ . In this regime,  $N \leq 2050$  and we find numerically that the number of logical cycles satisfies  $\mathcal{T} = 8 \times 10^6$ , which also upper bounds the  $T$  count. This implies that the logical spacetime volume satisfies  $N\mathcal{T} \leq 2 \times 10^{10}$ . Hence, the algorithm can be implemented with negligible failure probability provided the error rate per logical qubit and logical cycle satisfies  $p_L \ll 5 \times 10^{-11}$  and the  $|T\rangle$  state fidelity satisfies  $p_T \ll 10^{-7}$ . With reference to Table III, this is satisfied by using the  $d = 24$  GB code instantiation of the architecture for a physical error rate of  $p = 10^{-3}$  and the  $d = 10$  instantiation for  $p = 10^{-4}$ .

To implement this on the Pinnacle Architecture, we use a single processing unit with  $\beta = \lceil N/k \rceil$  processing blocks to ensure there are  $\kappa \geq N$  logical qubits available. We also account for a magic engine for this processing unit, but do not include memory. The required number of physical qubits is therefore given by

$$n = n_{pb} \left\lceil \frac{N}{k} \right\rceil + n_{me}. \quad (11)$$

Substituting  $N = 2L^2 + 2$  and the values for  $n_{pb}$ ,  $n_{me}$  and  $k$  from Section VB, the numbers of physical qubits required with physical error rates of  $p = 10^{-3}$  and  $p = 10^{-4}$ , respectively, are therefore

$$n_{10^{-3}} = 1620 \left\lceil \frac{L^2 + 1}{8} \right\rceil + 8694, \quad (12)$$

$$n_{10^{-4}} = 452 \left\lceil \frac{L^2 + 1}{6} \right\rceil + 2128. \quad (13)$$

Table IV shows the number of physical qubits required for even  $L \leq 32$ , along with the runtimes with code cycle times of  $1 \mu\text{s}$  and  $1 \text{ms}$ . Figure 2 shows that the number of physical qubits required is an order of magnitude smaller than those required using surface codes in Ref. [12].

A full determination of the ground state energy requires multiple shots of the algorithm, with the number

TABLE IV. Physical qubits and runtime required to perform one shot of Fermi-Hubbard ground state energy estimation on an  $L \times L$  lattice with a relative error of  $\leq 0.5\%$  of the total lattice energy. The runtime is approximately independent of  $L$  because the relative error allows for fewer Trotter steps for larger lattices; runtimes for each value of  $L$  are equal to or slightly smaller than those given. kq represents kiloqubits ( $\times 10^3$  qubits).

| Physical Qubits       |               |               |
|-----------------------|---------------|---------------|
| $L$                   | $p = 10^{-3}$ | $p = 10^{-4}$ |
| 8                     | 23 kq         | 7.1 kq        |
| 10                    | 30 kq         | 9.8 kq        |
| 12                    | 39 kq         | 13 kq         |
| 14                    | 49 kq         | 17 kq         |
| 16                    | 62 kq         | 22 kq         |
| 18                    | 75 kq         | 27 kq         |
| 20                    | 91 kq         | 32 kq         |
| 22                    | 108 kq        | 39 kq         |
| 24                    | 127 kq        | 46 kq         |
| 26                    | 146 kq        | 53 kq         |
| 28                    | 169 kq        | 61 kq         |
| 30                    | 192 kq        | 70 kq         |
| 32                    | 218 kq        | 79 kq         |
| Runtime (Upper Bound) |               |               |
| Code Cycle            | $p = 10^{-3}$ | $p = 10^{-4}$ |
| $1 \mu\text{s}$       | 3.6 min       | 1.6 min       |
| $1 \text{ms}$         | 2.5 days      | 1.1 days      |

dependent on the overlap between the initial state and the true ground state. These shots can be performed in series (with a proportional increase in run time) or could be performed in parallel using multiple separate processing units (with a proportional increase in the required physical qubits).

## B. RSA Factoring

In this subsection, we show that the Pinnacle Architecture can be used to efficiently perform the factoring necessary to break RSA encryption.

### 1. Algorithm

The algorithm we use is a generalisation of that presented by Gidney in Ref. [8], which uses techniques developed by Ekerå and Håstad [54] and by Chevignard et al. [55]. We refer to this algorithm as Gidney’s algorithm. This algorithm uses residue number system arithmetic to replace modular arithmetic over  $N_{\text{RSA}}$  (the number being factored) with modular arithmetic over a set of primes  $P$  that each have size polylogarithmic in  $N_{\text{RSA}}$ . This reduces the number of qubits required for the working register required for modular exponentiation (the dominant part of the factoring algorithm) from  $\Theta(\log N_{\text{RSA}})$  to  $\Theta(\log \log N_{\text{RSA}})$ , reducing the space overhead. For each individual prime, the time overhead for the modular exponentiation is also significantly reduced since the

time cost for arithmetic operations such as addition scales with the size of the input registers. However, in Gidney’s algorithm, this does not translate to an overall reduction in runtime because the  $|P|$  primes are processed in series.

We generalise Gidney’s algorithm by allowing for the possibility of processing multiple primes in parallel. This is done by replacing the working register with  $\rho$  copies of the register, for any positive integer  $\rho \leq |P|$ . The elements of  $P$  are partitioned into  $\rho$  subsets such that the maximum number of elements in each subset is  $\lceil |P|/\rho \rceil$ . The outer loop of Gidney’s algorithm is then applied in parallel on a different register for each subset. Finally, parallel reduction is used to combine the accumulators of each register into a single accumulator by aggregating accumulators pairwise in the form a binary tree. The final value of this accumulator matches that of Gidney’s algorithm, since it simply amounts to a reordering of the sum used to calculate the approximate modular exponential (Eq. (20) of Ref. [8]). The required result may therefore be extracted by measurement and classical processing in the same way. Letting  $\mathcal{T}_G$  be the time cost for Gidney’s algorithm, our algorithm has a time cost of

$$\mathcal{T} = \frac{\lceil |P|/\rho \rceil}{|P|} \mathcal{T}_G + O(\log \rho), \quad (14)$$

where the  $O(\log \rho)$  term accounts for the parallel reduction required to combine accumulators. Since  $|P|$  is large (e.g.,  $\approx 2.1 \times 10^4$  for RSA-2048 [8]), this can allow for a reduction in the time cost by many orders of magnitude.

Importantly, the additional space incurred by this form of parallelisation is not proportional to the reduction in time cost. To see why, note that the full register in Gidney’s algorithm has two components – an input register of  $m = \Theta(\log N_{\text{RSA}})$  logical qubits and a working register of  $N_w = \Theta(\log \log N_{\text{RSA}})$  logical qubits used to perform the approximate modular exponentiation. While the working register must be duplicated to allow for parallelisation, the same input register can be reused for many primes as it is only accessed by lookup operations. These operations commute for different primes since they all use only gates with controls on the input qubits. More precisely, given that a working register accesses the input register in windows of  $w_1$  logical qubits at a time, pipelining these accesses can allow  $\lceil m/w_1 \rceil$  registers to run in parallel using a single input register. Therefore, the required number of logical qubits is

$$N = \left\lceil \frac{\rho}{\lceil m/w_1 \rceil} \right\rceil m + \rho N_w. \quad (15)$$

Noting that  $\left\lceil \frac{\rho}{\lceil m/w_1 \rceil} \right\rceil = 1$  for  $\rho \lesssim 200$  and  $m$  is an order of magnitude larger than  $N_w$ , this can be significantly smaller than  $\rho N_1 = \rho(m + N_w)$  (where  $N_1$  is the number of logical qubits required for Gidney’s algorithm). This leads to significant spacetime savings from parallelisation, as shown in Fig. 7.

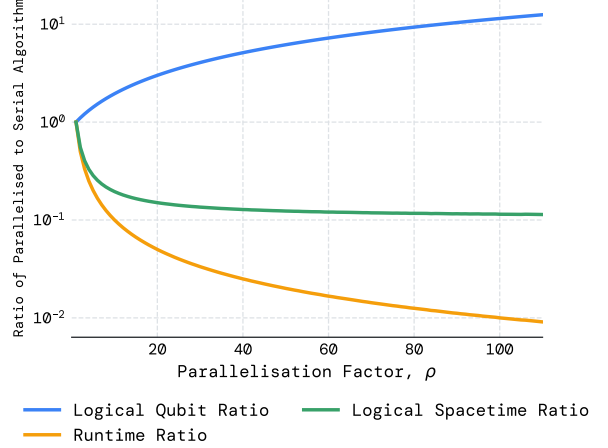


FIG. 7. Comparison of space, time and spacetime cost for our parallelised algorithm to Gidney’s algorithm or Ref. [8]. For the purpose of this plot, all algorithmic parameters (other than  $\rho$ ) are chosen to match those of the  $n = 2048$  row of Table 5 of Ref. [8].

Our modified algorithm allows for very efficient parallelism, with orders of magnitude reductions in the time overhead achievable with a smaller increase in the space overhead. In particular, we note that while Gidney’s algorithm has a longer runtime than the earlier implementation of Gidney and Ekerå [7], our parallelised version can achieve a significantly faster runtime. This motivates our choice of this algorithm even for architectures with slow clock cycles, where runtime is especially important.

## 2. Implementation on Pinnacle Architecture

To implement the algorithm on the Pinnacle Architecture, we begin by allocating a processing unit for each working register. Recall that the working registers act entirely independently for the entire duration of the modular exponentiations; they only interact at end when their accumulators are combined. This means that the processing units can run in parallel throughout the modular exponentiations, and then be joined pairwise as the parallel reduction is implemented. Since the joining happens at the end of the algorithm, it is not necessary to ever perform Clifford frame cleaning to separate the units again.

Each of these working registers is equipped with enough logical qubits to allow implementation of Gidney’s algorithm. This corresponds to: an  $f$  logical qubit sub-register for the overall accumulator; an  $\ell + \text{len}(m)$  logical qubit sub-register on which discrete-log values are accumulated for each prime; two ancillary sub-registers each with  $\max(f, \ell + \text{len}(m))$  logical qubits; a third ancillary sub-register with  $\ell$  logical qubits; and one additional ancillary logical qubit for compiling Toffoli gates from T gates using the circuit of Ref. [56]. As in Ref. [8],

$f$  is the length of the truncated accumulator,  $\ell$  is the bit length of the residue primes and  $\text{len}(m) = \lfloor \log_2(m) \rfloor + 1$  is the bit length of  $m$ . We therefore require  $\kappa$  logical qubits, given by

$$\kappa = f + 2\ell + \text{len}(m) + 2 \max(f, \ell + \text{len}(m)) + 1. \quad (16)$$

To achieve this, we allocate  $\lceil \kappa/k \rceil$  processing blocks to each processing unit, where  $k$  is the number of logical qubits per code block.

The input register (or multiple input registers if  $\rho \geq \lceil m/w_1 \rceil$ ) is associated with memory in the architecture. Access to this memory occurs during loop 1 of Gidney's algorithm, when windows of  $w_1$  logical qubits are used as the address for a lookup operation targeted on each working register. As discussed in Section VB3, we fix  $w_1 = k/2$ , and we also enforce the condition that  $\ell \geq w_1$ . This ensures that there are at least  $w_1$  unused logical qubits onto which the window can be fanned out, since at the time of this loop,  $f + 3\ell + 3 \text{len}(m)$  logical qubits of each working register are in use, meaning that there are at least  $\ell$  unused logical qubits.

Otherwise, we follow the decomposition into addition, lookup and phaseup subroutines and implementation of these subroutines presented in Ref. [8].

### 3. Resource Analysis

*a. Physical Qubits:* We now determine the number of physical qubits required. Each working register corresponds to a processing unit with  $\kappa(f, \ell, m)$  logical qubits, as given in Eq. (16), along with a magic engine. The number of physical qubits required for the  $\rho$  working registers is therefore

$$n_w = \rho \left( n_{pb} \left\lceil \frac{\kappa(f, \ell, m)}{k} \right\rceil + n_{me} \right). \quad (17)$$

Each memory stores  $m$  logical qubits and there are  $\lceil \frac{\rho}{\lceil m/w_1 \rceil} \rceil$  such memories. Each memory requires  $\nu = \lceil m/k \rceil$  code blocks of an  $[[n, k, d]]$  code (with  $2n$  physical qubits each). For each processing unit, we also require a port with  $n_g + n_b$  additional physical qubits. The number of physical qubits required for the memory and ports is therefore

$$n_m = 2n \left\lceil \frac{\rho}{\lceil m/w_1 \rceil} \right\rceil \left\lceil \frac{m}{k} \right\rceil + \rho(n_g + n_b). \quad (18)$$

Hence the total number of physical qubits is  $n_{\text{total}} = n_w + n_m$ .

*b. Runtime:* We define the total number of logical cycles required per iteration of the outer loop of Gidney's algorithm to be  $\Sigma$ , which is equal to the sum of the final column of Table V. Since we require  $\lceil |P|/\rho \rceil$  iterations, this implies that the expected number of logical cycles required for the full outer loop is no more than  $\lceil |P|/\rho \rceil \Sigma$ .

Following the completion of this outer loop, we have some further minor steps that are performed once. First, the uncomputation of loop 1 [8]—which is performed in parallel on all streams—consists of a single instance of the loop 1 lookup and addition. Referring to Table V, it takes  $v = \lceil m/w_1 \rceil (6(2^{w_1} - w_1 + \ell + \text{len}(m) - 2) + 2w_1)$  logical cycles. Second, the accumulators for the different streams must be combined, which takes  $\lceil \log_2(\rho) \rceil$  layers of additions of  $f$  qubit registers and hence  $6(f - 1)\lceil \log_2(\rho) \rceil$  logical cycles. Finally, there is a frequency measurement (i.e. inverse quantum Fourier transform followed by measurement); the number of logical cycles required for this is negligible compared to  $\Sigma$  so we follow Ref. [8] in omitting it from our accounting. Assuming a perfect success rate of magic engines we can therefore write the total number of logical cycles as

$$\mathcal{T}' = \left( \left\lceil \frac{|P|}{\rho} \right\rceil \Sigma + v + 6(f - 1)\lceil \log_2(\rho) \rceil \right). \quad (19)$$

The proportion of logical cycles requiring  $T$  states is approximately (in fact, slightly less than)  $2/3$ , which leads to an adjusted formula for the true number of logical cycles accounting for the magic engine rejection rate of  $p_r$  of

$$\mathcal{T} = \left( \frac{2}{3}(1 - p_r)^{-1} + \frac{1}{3} \right) \mathcal{T}' \quad (20)$$

Then the total runtime per shot  $t = t_l \mathcal{T}$  is the product of the number of logical cycles  $\mathcal{T}$  and the time per logical cycle  $t_l = d_t t_c$ .

The expected number of shots required for the factoring, from Ref. [8], is given by

$$\sigma = \frac{s + 1}{0.99 p_S \left( 1 - 2N_{\text{RSA}} \sqrt{\frac{s+2}{2^{f+1} s w_1}} \right)}, \quad (21)$$

where  $s$  is the Ekera-Håstad parameter, and  $p_S$  is the probability that a shot does not have a logical error, which is given by

$$p_S = (1 - p_L)^{N\mathcal{T}} (1 - p_T)^\tau, \quad (22)$$

where  $p_L$  is the logical error rate per logical qubit per logical cycle,  $p_T$  is the infidelity of output  $T$  states from the magic engine,  $N$  and  $\mathcal{T}$  are the total number of logical qubits and logical cycles per shot and  $\tau$  is the  $T$  count per shot, which is given by

$$\tau = |P|\tau_1 + \frac{2}{3}v + 4(f - 1)\lceil \log_2(\rho) \rceil, \quad (23)$$

where  $\tau_1$  is the  $T$  count per prime which is equal to the sum of the  $T$  count column of Table V. Hence the expected runtime for the factoring is  $t_{\text{total}} = \sigma t$ .

TABLE V. Time cost accounting for all subroutines required for each prime. Loop numbers refer to Gidney’s algorithm, as presented in Ref. [8]. For lookups and phaseups, the size is the address size; for additions, it is the size of the registers being added. For addition (loop 2) the size is the average over the loop, since it varies. The  $T$  Count column is the number of  $T$  gates required, which is four times the Toffoli count. The Logical Cycles column is the total number of logical cycles, including both  $T$  state injections and other logical measurements. Since each Toffoli gate requires four  $T$  gates and one logical measurement [56], and one additional logical measurement is required for measurement-based uncompute, this is nearly always  $3/2$  times the  $T$  count. The only exception is Lookup (Loop 1) which also requires additional logical measurements for Clifford frame cleaning.

| Subroutine         | Size                            | Instances                                                 | $T$ Count                                                                                                              | Logical Cycles                                                                                                         |
|--------------------|---------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|
| Lookup (Loop 1)    | $w_1$                           | $\lceil m/w_1 \rceil$                                     | $4\lceil m/w_1 \rceil (2^{w_1} - w_1 - 1)$                                                                             | $\lceil m/w_1 \rceil (6(2^{w_1} - w_1 - 1) + 2w_1)$                                                                    |
| Addition (Loop 1)  | $\ell + \text{len}(m)$          | $\lceil m/w_1 \rceil$                                     | $4\lceil m/w_1 \rceil (\ell + \text{len}(m) - 1)$                                                                      | $6\lceil m/w_1 \rceil (\ell + \text{len}(m) - 1)$                                                                      |
| Addition (Loop 2)  | $(2\ell + \text{len}(m) + 1)/2$ | $4\text{len}(m)$                                          | $8\text{len}(m) (2\ell + \text{len}(m) - 1)$                                                                           | $12\text{len}(m) (2\ell + \text{len}(m) - 1)$                                                                          |
| Lookup (Loop 3)    | $2w_3$                          | $4\lceil \ell/w_3 \rceil^2 - 8\lceil \ell/w_3 \rceil + 1$ | $4(4\lceil \ell/w_3 \rceil^2 - 8\lceil \ell/w_3 \rceil + 1)(2^{2w_3} - 2w_3 - 1)$                                      | $6(4\lceil \ell/w_3 \rceil^2 - 8\lceil \ell/w_3 \rceil + 1)(2^{2w_3} - 2w_3 - 1)$                                      |
| Addition (Loop 3)  | $\ell$                          | $7\lceil \ell/w_3 \rceil^2 - 14\lceil \ell/w_3 \rceil$    | $28(\lceil \ell/w_3 \rceil^2 - 2\lceil \ell/w_3 \rceil)(\ell - 1)$                                                     | $42(\ell - 1)(\lceil \ell/w_3 \rceil^2 - 2\lceil \ell/w_3 \rceil)$                                                     |
| Lookup (Loop 4)    | $w_4$                           | $3\lceil \ell/w_4 \rceil/2$                               | $6\lceil \ell/w_4 \rceil (2^{w_4} - w_4 - 1)$                                                                          | $9\lceil \ell/w_4 \rceil (2^{w_4} - w_4 - 1)$                                                                          |
| Addition (Loop 4)  | $f$                             | $5\lceil \ell/w_4 \rceil/2$                               | $10(f - 1)\lceil \ell/w_4 \rceil$                                                                                      | $15(f - 1)\lceil \ell/w_4 \rceil$                                                                                      |
| Phaseup (Loop 4)   | $w_4$                           | $\lceil \ell/w_4 \rceil$                                  | $4\lceil \ell/w_4 \rceil (2^{\lceil w_4/2 \rceil} + 2^{\lfloor w_4/2 \rfloor} - w_4 - 2)$                              | $6\lceil \ell/w_4 \rceil (2^{\lceil w_4/2 \rceil} + 2^{\lfloor w_4/2 \rfloor} - w_4 - 2)$                              |
| Phaseup (Loop 3.2) | $w_3$                           | $3\lceil \ell/w_3 \rceil^2/2 - 3\lceil \ell/w_3 \rceil$   | $6(\lceil \ell/w_3 \rceil^2 - 2\lceil \ell/w_3 \rceil)(2^{\lceil w_3/2 \rceil} + 2^{\lfloor w_3/2 \rfloor} - w_3 - 2)$ | $9(\lceil \ell/w_3 \rceil^2 - 2\lceil \ell/w_3 \rceil)(2^{\lceil w_3/2 \rceil} + 2^{\lfloor w_3/2 \rfloor} - w_3 - 2)$ |
| Phaseup (Loop 3.1) | $2w_3$                          | 1                                                         | $4(2^{w_3+1} - 2w_3 - 2)$                                                                                              | $6(2^{w_3+1} - 2w_3 - 2)$                                                                                              |

#### 4. Results

We now consider the resources—both physical qubits and time—required to factor an RSA-2048 integer on the instantiation of the Pinnacle architecture presented in Section V, given different hardware parameters, namely the code cycle time and physical error rate. Following Ref. [8], we expect the required logical error rate per logical qubit per logical cycle to be  $\lesssim 10^{-14}$  (i.e.,  $\lesssim 10^{-15}$  per code cycle). With reference to Table III, this motivates a choice of the  $d = 24$  GB code architecture for a physical error rate of  $p = 10^{-3}$  and  $d = 16$  for the  $p = 10^{-4}$  architecture. The precise logical failure rate of the algorithm varies somewhat as other parameters affect the number of logical qubits and logical cycles; this effect is accounted for in the number of shots, given in Eq. (21).

To determine the minimal required physical qubits and runtime, we optimise over the algorithmic parameters of Gidney’s algorithm with the following ranges:

- Ekerå-Håstad parameter,  $1 \leq s \leq 16$ ;
- Accumulator truncation,  $24 \leq f \leq 59$ ;
- Prime bit length,  $18 \leq \ell \leq 25$ ;
- Loop 3 window size,  $2 \leq w_3 \leq 6$ ;
- Loop 4 window size,  $2 \leq w_4 \leq 6$ .

We also optimise the parallelisation factor over the range  $1 \leq \rho \leq |P|$ , where  $|P| \approx nm/\ell w_1$  is the number of primes in the residue system. Recall that, unlike in Ref. [8], we fix the loop 1 window size as  $w_1 = k/2$ . Following Ref. [8], we also impose the feasibility condition that the number of primes of bit length  $\ell$ ,  $\pi(\ell) \approx 2^{\ell-1}/\ell \ln(2)$ , cannot be smaller than the number of primes  $|P|$ . The results of this optimisation are shown in Table VI and Fig. 3.

We find that fewer than one hundred thousand physical qubits are required for factoring at a physical error rate of

TABLE VI. Minimum number of physical qubits required to complete factoring in a range of expected runtimes for a range of hardware parameters. Mq and kq represent megaqubits (i.e.,  $\times 10^6$  qubits) and kiloqubits ( $\times 10^3$  qubits) respectively.

| Code Cycle  | Physical Error Rate | Physical Qubits for Runtime $\leq$ |         |        |        |
|-------------|---------------------|------------------------------------|---------|--------|--------|
|             |                     | 1 year                             | 1 month | 1 week | 1 day  |
| 1 $\mu$ s   | $10^{-3}$           | 97 kq                              | 98 kq   | 151 kq | 471 kq |
|             | $10^{-4}$           | 53 kq                              | 53 kq   | 65 kq  | 149 kq |
| 10 $\mu$ s  | $10^{-3}$           | 97 kq                              | 223 kq  | 624 kq | 3.8 Mq |
|             | $10^{-4}$           | 53 kq                              | 85 kq   | 184 kq | 949 kq |
| 100 $\mu$ s | $10^{-3}$           | 198 kq                             | 1.3 Mq  | 5.4 Mq | 41 Mq  |
|             | $10^{-4}$           | 79 kq                              | 359 kq  | 1.3 Mq | 9.5 Mq |
| 1 ms        | $10^{-3}$           | 1.1 Mq                             | 13 Mq   | 58 Mq  | -      |
|             | $10^{-4}$           | 307 kq                             | 3.1 Mq  | 14 Mq  | 128 Mq |

$p = 10^{-3}$  in an expected runtime of one month. Alternatively, with the same error rate and code cycle time, factoring can be completed in one week with 151 thousand physical qubits—compared to one million in Ref. [8]—or in one day with 471 thousand physical qubits. With one million physical qubits, factoring takes an expected time of ten hours, compared with five days in Ref. [8].

We can also consider our results in regimes relevant to other hardware platforms. For example, at a physical error rate of  $p = 10^{-4}$ —relevant to trapped ions [15]—the minimum number of physical qubits required for factoring is 53 thousand. For a typical trapped ion code cycle time of 1 ms [57], factoring with one million physical qubits takes an expected time of approximately three months. Factoring in a shorter time with these parameters is also possible with a feasible number of physical qubits. For example, factoring can be completed in one month with 3.1 million physical qubits or in one week with 14 million physical qubits. By comparison, adjusting the results of Ref. [8] to such a code cycle time would give a prohibitively long runtime, while Ref. [58] estimated a runtime of 3 years



with 8.6 million qubits for a surface code architecture with trapped ion parameters.

Our results are also comparable to the results of Ref. [16], which apply to a neutral atom platform with physical error rate of  $p = 10^{-3}$  and a code cycle time of 1 ms. Specifically, with 19 million qubits we find a runtime of 21 days, compared with the result of 5.6 days in Ref. [16]. The low runtime achieved by that work is achieved by using transversal gates with algorithmic fault tolerance to reduce the logical cycle time to be equal to the code cycle time, compared with  $d_t = 26$  code cycles on our architecture. Our architecture has the potential to support an analogous reduction by incorporating fast surgery [59], with the potential to achieve an order-of-magnitude lower runtime or physical qubit number. We look forward to realising this potential in future work.

## VII. CONCLUSION

We have presented the Pinnacle Architecture, which leverages the high encoding rate of QLDPC codes to achieve universal quantum computing with order-of-magnitude overhead reductions compared with surface code architectures. In particular, we have shown that factoring 2048-bit RSA integers, which requires close to a million physical qubits using surface code architectures [8], can be done with fewer than one hundred thousand physical qubits on the Pinnacle Architecture. Given the challenges posed in scaling from one hundred thousand to one million physical qubits, such as the need on many hardware platforms for networking between separated devices [9], this has the potential to significantly hasten the onset of practical quantum computing.

Importantly, this is only the beginning of the story for QLDPC architectures. While it has been suggested that reducing the physical qubit count for 2048-bit RSA factoring by an order of magnitude (i.e., to one hundred thousand physical qubits) is implausible on surface code architectures [8], the same cannot be said of QLDPC architectures. Indeed, a range of higher rate QLDPC codes are known than the generalised bicycle codes used here [18]. Incorporating such codes into the Pinnacle Architecture, combined with further optimisation of its components, could plausibly achieve such a reduction. As such, this work serves not only as a major step forward in its own right, but also a foundation on which we expect to make substantial further progress.

## VIII. ACKNOWLEDGEMENTS

We thank Stephen Bartlett and Kevin Obenland for discussions, Calida Tang for organisational support, Fernando Borretti for software engineering support, Max McIsted for assistance with preparing the figures, and Scott Aaronson for thoughtful feedback on the title.

## Appendix A: Cost of Clifford Frame Cleaning

In this appendix, we show that Clifford frame cleaning can be completed using the number of logical measurements claimed in Section IV B 3. Specifically, we show that it can be completed in the same number of Pauli  $\pi/4$  rotations, which we write as  $R_{\pi/4}(P)$  for a Pauli rotation axis  $P$ . Each such rotation can be implemented by a joint logical Pauli measurement with a single logical ancilla in the  $|\bar{0}\rangle$  state [6].

We present our proofs using the Pauli and Clifford operator representation of Ref. [60]. Specifically, an  $n$ -qubit Pauli operator is associated (uniquely, up to a global phase) with an element  $v \in \mathbb{Z}_2^{2n}$  by the expression  $P_v = \prod_{i=1}^n X_i^{v_i} Z_i^{v_{n+i}}$ . In this formalism,  $P_u P_v = P_{u+v}$ . Commutation relations are specified by the symplectic inner product

$$\langle u, v \rangle = u J v^T, \quad J = \begin{bmatrix} 0 & I_n \\ I_n & 0 \end{bmatrix}, \quad (\text{A1})$$

and  $u$  and  $v$  are treated as row vectors. Specifically,  $P_u$  and  $P_v$  commute if  $\langle u, v \rangle = 0$ , and anti-commute if  $\langle u, v \rangle = 1$ . The symplectic complement  $W^\perp$  of subspace  $W$  is a subspace whose elements commute with all elements in  $W$ .

In the same formalism, an  $n$ -qubit Clifford operator  $U$  can be represented by a  $2n \times 2n$  matrix  $M_U$  that preserves the symplectic inner product (i.e., such that  $M_U J M_U^T = J$ ). The action of  $U$  on  $P_v$  by conjugation corresponds to matrix multiplication on the right (i.e.,  $P \mapsto U P U^\dagger$  corresponds to  $v \mapsto v M_U$ ). The product of two Clifford operators  $VU$  therefore corresponds to the product of their matrix representations  $M_U M_V$ . A Pauli  $\pi/4$  rotation  $R_{\pi/4}(Q)$  is a special type of Clifford operator which is specified by an  $n$ -qubit Pauli operator  $Q$ . It acts as  $R_{\pi/4}(Q) P R_{\pi/4}(Q)^\dagger = PQ$  if  $P$  and  $Q$  anti-commute, and as  $R_{\pi/4}(Q) P R_{\pi/4}(Q)^\dagger = P$  if  $P$  and  $Q$  commute. Therefore,  $R_{\pi/4}(P_u)$  acts on  $v \in \mathbb{Z}_2^{2n}$  as  $E_u(v) = v + \langle u, v \rangle u$ .

There are two cases of Clifford frame cleaning to consider: the general case, where  $4w$  steps are required to clean off  $w$  qubits; and the case of cleaning a memory port, where only  $2w$  steps are required. We first consider the general case.

**Lemma 1.** Let  $U$  be an  $n$ -qubit Clifford operator. Then for  $w \leq n$ , there exists a sequence of  $4w$  Pauli operators  $P_1, \dots, P_{4w}$  such that  $R_{\pi/4}(P_1) R_{\pi/4}(P_2) \dots R_{\pi/4}(P_{4w}) U$  is a Clifford operator supported only on the last  $n - w$  qubits.

*Proof.* We proceed by induction. Let

$$M^{(k)} = \left[ \begin{array}{cc|cc} I_k & 0 & 0 & 0 \\ 0 & * & 0 & * \\ \hline 0 & 0 & I_k & 0 \\ 0 & * & 0 & * \end{array} \right]. \quad (\text{A2})$$



We will show for  $1 \leq k \leq w$ , given a matrix of the form  $M^{(k-1)}$ , that there exists a product of four Pauli  $\pi/4$  rotations with matrices  $E_{\alpha_1}$ ,  $E_{\alpha_2}$ ,  $E_{\alpha_3}$ , and  $E_{\alpha_4}$ , such that  $E_{\alpha_4}E_{\alpha_3}E_{\alpha_2}E_{\alpha_1}M^{(k-1)}$  is of the form  $M^{(k)}$ . It follows that  $U = M^{(0)}$  can be mapped to an operator  $M^{(w)}$  that has support only on the last  $n - w$  qubits with  $4w$  Pauli  $\pi/4$  rotations.

Let  $v^{(k)}$  denote the  $k$ th row of  $M^{(k-1)}$ ,  $e_k$  denote the  $k$ th vector in the standard symplectic basis and  $f_k = e_{n+k}$ , and let  $A_{k-1}$  be the symplectic subspace spanned by  $\{e_1, \dots, e_{k-1}, f_1, \dots, f_{k-1}\}$  corresponding to the Pauli group on the first  $k - 1$  qubits. We will choose  $\alpha_1, \alpha_2, \alpha_3, \alpha_4 \in A_{k-1}^\perp$  to ensure that these operations act as the identity on all  $a \in A_{k-1}$ . We now consider two cases, and in each choose  $\alpha_1, \alpha_2$  such that they map  $v^{(k)}$  to  $e_k$ .

First, if  $\langle v^{(k)}, e_k \rangle = 1$ , let  $\alpha_1 = v^{(k)} + e_k$  and  $\alpha_2 = 0$ . Then  $\langle v^{(k)}, v^{(k)} + e_k \rangle = 1$ , and so

$$E_{v^{(k)}+e_k}(v^{(k)}) = e_k. \quad (\text{A3})$$

Second, if  $\langle v^{(k)}, e_k \rangle = 0$ , then if  $\langle v^{(k)}, f_k \rangle = 1$  let  $u_k = f_k$ , and if  $\langle v^{(k)}, f_k \rangle = 0$  let  $u_k = f_k + e_{n+\delta \pmod{2n}}$ , where  $\delta > k$  is the position of the first nonzero element of  $v^{(k)}$ . Moreover, let  $\alpha_1 = v^{(k)} + u_k$  and  $\alpha_2 = e_k + u_k$ . Then  $\langle v^{(k)}, v^{(k)} + u_k \rangle = \langle u_k, e_k + u_k \rangle = 1$ , and so

$$E_{e_k+u_k}E_{v^{(k)}+u_k}(v^{(k)}) = E_{e_k+u_k}(u_k) = e_k. \quad (\text{A4})$$

Let  $\tilde{M}^{(k-1)} = E_{e_k+u}E_{v^{(k)}+u}(M^{(k-1)})$  and the  $i$ th row of  $\tilde{M}^{(k-1)}$  be  $\tilde{v}^{(i)}$ . As this preserves the symplectic product,  $1 = \langle v^{(k)}, v^{(n+k)} \rangle = \langle \tilde{v}^{(k)}, \tilde{v}^{(n+k)} \rangle = \langle e_k, \tilde{v}^{(n+k)} \rangle$ . We now consider two cases, and in each choose  $\alpha_3, \alpha_4$  such that they fix  $e_k$  and map  $\tilde{v}^{(n+k)}$  to  $f_k$ .

First, if  $\langle \tilde{v}^{(n+k)}, f_k \rangle = 1$ , let  $\alpha_3 = \tilde{v}^{(n+k)} + f_k$  and  $\alpha_4 = 0$ . Then  $\langle \tilde{v}^{(n+k)}, \tilde{v}^{(n+k)} + f_k \rangle = 1$ , and so

$$E_{\tilde{v}^{(n+k)}+f_k}(\tilde{v}^{(n+k)}) = f_k. \quad (\text{A5})$$

We also have  $\langle e_k, \tilde{v}^{(n+k)} + f_k \rangle = 0$ , which implies that  $E_{\tilde{v}^{(n+k)}+f_k}(e_k) = e_k$ .

Second, if  $\langle \tilde{v}^{(n+k)}, f_k \rangle = 0$ , let  $\alpha_3 = \tilde{v}^{(n+k)} + e_k + f_k$  and  $\alpha_4 = e_k$ . Then  $\langle \tilde{v}^{(n+k)}, \tilde{v}^{(n+k)} + e_k + f_k \rangle = 1$ , and so

$$E_{e_k}E_{\tilde{v}^{(n+k)}+e_k+f_k}(\tilde{v}^{(n+k)}) = E_{e_k}(e_k + f_k) = f_k. \quad (\text{A6})$$

We also have  $\langle e_k, \tilde{v}^{(n+k)} + e_k + f_k \rangle = 0$ , which implies that  $E_{\tilde{v}^{(n+k)}+e_k+f_k}(e_k) = e_k$ .

Hence in all cases

$$M^{(k)} = E_{\alpha_4}E_{\alpha_3}E_{\alpha_2}E_{\alpha_1}M^{(k-1)}, \quad (\text{A7})$$

which completes the proof.  $\square$

Now we consider the specific case of cleaning a memory port.

**Lemma 2.** Let  $U$  be an  $n$ -qubit Clifford operator that is a product of Clifford operators that act either trivially or as the control of a CNOT on the first  $w \leq n$  qubits. Then

there exists a sequence of  $2w$  Pauli operators  $P_1, \dots, P_{2w}$  such that  $R_{\pi/4}(P_1)R_{\pi/4}(P_2) \dots R_{\pi/4}(P_{2w})U$  is a Clifford operator supported only on the last  $n - w$  qubits.

*Proof.* Since CNOTs commute with  $Z$  operators on their control qubits,  $U$  must commute with all  $Z$ -type operators with support on the first  $w$  qubits. Consequently, the matrix representation of  $U$  has the form

$$U = \left[ \begin{array}{cc|cc} I_w & * & * & * \\ 0 & * & * & * \\ \hline 0 & 0 & I_w & 0 \\ 0 & * & * & * \end{array} \right]. \quad (\text{A8})$$

We proceed by induction, analogously to the proof of Lemma 1. Let

$$M^{(k)} = \left[ \begin{array}{ccc|ccc} I_k & 0 & 0 & 0 & 0 & 0 \\ 0 & I_{w-k} & * & 0 & * & * \\ 0 & 0 & * & 0 & * & * \\ \hline 0 & 0 & 0 & I_k & 0 & 0 \\ 0 & 0 & 0 & 0 & I_{w-k} & 0 \\ 0 & 0 & * & 0 & * & * \end{array} \right]. \quad (\text{A9})$$

We will show for  $1 \leq k \leq w$ , given a matrix of the form  $M^{(k-1)}$ , that there exists a product of two Pauli  $\pi/4$  rotations with matrices  $E_{\alpha_1}$  and  $E_{\alpha_2}$ , such that  $E_{\alpha_2}E_{\alpha_1}M^{(k-1)}$  is of the form  $M^{(k)}$ . It follows that  $U = M^{(0)}$  can be mapped to an operator  $M^{(w)}$  that has support only on the last  $n - w$  qubits with  $2w$  Pauli  $\pi/4$  rotations.

Let  $v^{(k)}$  denote the  $k$ th row of  $M^{(k-1)}$ ,  $e_k$  denote the  $k$ th vector in the standard symplectic basis and  $f_k = e_{n+k}$ , and let  $A_{k-1}$  be the symplectic subspace spanned by  $\{e_1, \dots, e_{k-1}, f_1, \dots, f_{k-1}\}$  corresponding to the Pauli group on the first  $k - 1$  qubits. We will choose  $\alpha_1, \alpha_2 \in A_{k-1}^\perp$  to ensure that these operations act as the identity on all  $a \in A_{k-1}$ , and such that they also fix  $f_j$  for  $k \leq j \leq w$  and map  $v^{(k)}$  to  $e_k$ . We now consider two cases, noting that the structure of  $M^{(k-1)}$  implies that  $\langle v^{(k)}, f_j \rangle = \delta_{jk}$  for  $1 \leq j \leq w$ .

First, if  $\langle v^{(k)}, e_k \rangle = 1$ , let  $\alpha_1 = v^{(k)} + e_k$  and  $\alpha_2 = 0$ . Then  $\langle v^{(k)}, v^{(k)} + e_k \rangle = 1$ , and so

$$E_{v^{(k)}+e_k}(v^{(k)}) = e_k. \quad (\text{A10})$$

We also have  $\langle f_j, v^{(k)} + e_k \rangle = 2\delta_{jk} = 0$  for  $k \leq j \leq w$ , which implies that  $E_{v^{(k)}+e_k}(f_j) = f_j$ .

Second, if  $\langle v^{(k)}, e_k \rangle = 0$ , let  $\alpha_1 = v^{(k)} + e_k + f_k$  and  $\alpha_2 = f_k$ . Then  $\langle v^{(k)}, v^{(k)} + e_k + f_k \rangle = 1$ , and so

$$E_{f_k}E_{v^{(k)}+e_k+f_k}(v^{(k)}) = E_{f_k}(e_k + f_k) = e_k. \quad (\text{A11})$$

We also have  $\langle f_j, v^{(k)} + e_k \rangle = 2\delta_{jk} = 0$  for  $k \leq j \leq w$ , which implies that  $E_{f_k}E_{v^{(k)}+e_k+f_k}(f_j) = f_j$ .

Hence in all cases

$$M^{(k)} = E_{\alpha_2}E_{\alpha_1}M^{(k-1)}, \quad (\text{A12})$$

which completes the proof.  $\square$

- 
- [1] P. Shor, Algorithms for quantum computation: Discrete logarithms and factoring, in *Proceedings 35th Annual Symposium on Foundations of Computer Science* (1994) pp. 124–134.
- [2] S. Lloyd, Universal quantum simulators, *Science* **273**, 1073 (1996).
- [3] D. Gottesman, Opportunities and challenges in fault-tolerant quantum computation, Preprint (2022), [arXiv:2210.15844](#).
- [4] A. G. Fowler, M. Mariantoni, J. M. Martinis, and A. N. Cleland, Surface codes: Towards practical large-scale quantum computation, *Physical Review A* **86**, 032324 (2012), [arXiv:1208.0928](#).
- [5] D. Horsman, A. G. Fowler, S. Devitt, and R. V. Meter, Surface code quantum computing by lattice surgery, *New Journal of Physics* **14**, 123011 (2012), [arXiv:1111.4022](#).
- [6] D. Litinski, A game of surface codes: Large-scale quantum computing with lattice surgery, *Quantum* **3**, 128 (2019), [arXiv:1808.02892](#).
- [7] C. Gidney and M. Ekerå, How to factor 2048 bit RSA integers in 8 hours using 20 million noisy qubits, *Quantum* **5**, 433 (2021), [arXiv:1905.09749](#).
- [8] C. Gidney, How to factor 2048 bit RSA integers with less than a million noisy qubits, Preprint (2025), [arXiv:2505.15917](#).
- [9] M. Mohseni, A. Scherer, K. G. Johnson, O. Wertheim, M. Otten, N. A. Aadit, Y. Alexeev, K. M. Bresniker, K. Y. Cam-sari, B. Chapman, S. Chatterjee, G. A. Dagnew, A. Esposito, F. Fahim, M. Fiorentino, A. Gajjar, A. Khalid, X. Kong, B. Kulchytsky, E. Kyoseva, R. Li, P. A. Lott, I. L. Markov, R. F. McDermott, G. Pedretti, P. Rao, E. Rieffel, A. Silva, J. Sorebo, P. Spentzouris, Z. Steiner, B. Torosov, D. Venturelli, R. J. Visser, Z. Webb, X. Zhan, Y. Cohen, P. Ronagh, A. Ho, R. G. Beausoleil, and J. M. Martinis, How to build a quantum supercomputer: Scaling from hundreds to millions of qubits, Preprint (2025), [arXiv:2411.10406](#).
- [10] T. J. Yoder, E. Schoute, P. Rall, E. Pritchett, J. M. Gambetta, A. W. Cross, M. Carroll, and M. E. Beverland, Tour de gross: A modular quantum computer based on bivariate bicycle codes, Preprint (2025), [arXiv:2506.03094](#).
- [11] P. Webster, S. C. Smith, and L. Z. Cohen, Explicit construction of low-overhead gadgets for gates on quantum LDPC codes, Preprint (2025), [arXiv:2511.15989](#).
- [12] I. D. Kivlichan, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, W. Sun, Z. Jiang, N. Rubin, A. Fowler, A. Aspuru-Guzik, H. Neven, and R. Babbush, Improved fault-tolerant quantum simulation of condensed-phase correlated electrons via Trotterization, *Quantum* **4**, 296 (2020), [arXiv:1902.10673](#).
- [13] S. Bravyi, G. Smith, and J. Smolin, Trading classical and quantum computational resources, *Physical Review X* **6**, 021043 (2016), [arXiv:1506.01396](#).
- [14] E. T. Campbell, Early fault-tolerant simulations of the Hubbard model, *Quantum Science and Technology* **7**, 015007 (2022), [arXiv:2012.09238](#).
- [15] A. C. Hughes, R. Srinivas, C. M. Löschnauer, H. M. Knaack, R. Matt, C. J. Ballance, M. Malinowski, T. P. Harty, and R. T. Sutherland, Trapped-ion two-qubit gates with >99.99% fidelity without ground-state cooling, Preprint (2025), [arXiv:2510.17286](#).
- [16] H. Zhou, C. Duckering, C. Zhao, D. Bluvstein, M. Cain, A. Kubica, S.-T. Wang, and M. D. Lukin, Resource analysis of low-overhead transversal architectures for reconfigurable atom arrays, in *Proceedings of the 52nd Annual International Symposium on Computer Architecture* (2025) pp. 1432–1448, [arXiv:2505.15907](#).
- [17] D. J. C. MacKay, G. Mitchison, and P. L. McFadden, Sparse graph codes for quantum error-correction, *IEEE Transactions on Information Theory* **50**, 2315 (2004), [arXiv:quant-ph/0304161](#).
- [18] N. P. Breuckmann and J. N. Eberhardt, Quantum low-density parity-check codes, *PRX Quantum* **2**, 040101 (2021), [arXiv:2103.06309](#).
- [19] J. Yoneda, W. Huang, M. Feng, C. H. Yang, K. W. Chan, T. Tanttu, W. Gilbert, R. C. C. Leon, F. E. Hudson, K. M. Itoh, A. Morello, S. D. Bartlett, A. Laucht, A. Saraiva, and A. S. Dzurak, Coherent spin qubit transport in silicon, *Nature Communications* **12**, 4114 (2021), [arXiv:2008.04020](#).
- [20] M. Malinowski, D. T. C. Allcock, and C. J. Ballance, How to wire a 1000-qubit trapped ion quantum computer, *PRX Quantum* **4**, 040313 (2023), [arXiv:2305.12773](#).
- [21] D. Bluvstein, H. Levine, G. Semeghini, T. T. Wang, S. Ebadi, M. Kalinowski, A. Keesling, N. Maskara, H. Pichler, M. Greiner, V. Vuletic, and M. D. Lukin, A quantum processor based on coherent transport of entangled atom arrays, *Nature* **604**, 451 (2022), [arXiv:2112.03923](#).
- [22] H. Bombin, I. H. Kim, D. Litinski, N. Nickerson, M. Pant, F. Pastawski, S. Roberts, and T. Rudolph, Interleaving: Modular architectures for fault-tolerant photonic quantum computing, Preprint (2021), [arXiv:2103.08612](#).
- [23] S. Bravyi, A. W. Cross, J. M. Gambetta, D. Maslov, P. Rall, and T. J. Yoder, High-threshold and low-overhead fault-tolerant quantum memory, *Nature* **627**, 778 (2024), [arXiv:2308.07915](#).
- [24] L. Z. Cohen, I. H. Kim, S. D. Bartlett, and B. J. Brown, Low-overhead fault-tolerant quantum computing using long-range connectivity, *Science Advances* **8**, eabn1717 (2022), [arXiv:2110.10794](#).
- [25] D. J. Williamson and T. J. Yoder, Low-overhead fault-tolerant quantum computation by gauging logical operators, Preprint (2024), [arXiv:2410.02213](#).
- [26] B. Ide, M. G. Gowda, P. J. Nadkarni, and G. Dauphinais, Fault-tolerant logical measurements via homological measurement, *Physical Review X* **15**, 021088 (2025), [arXiv:2410.02753](#).
- [27] Following Refs [6, 10], we assume that the time required between logical measurements is negligible compared with the timescale of a logical cycle.
- [28] A. W. Cross, Z. He, P. J. Rall, and T. J. Yoder, Improved QLDPC surgery: Logical measurements and bridging codes, Preprint (2025), [arXiv:2407.18393](#).
- [29] E. Swaroop, T. Jochym-O'Connor, and T. J. Yoder, Universal adapters between quantum LDPC codes, Preprint (2025), [arXiv:2410.03628](#).
- [30] R. Acharya, L. Aghababaie-Beni, I. Aleiner, T. I. Andersen, M. Ansmann, F. Arute, K. Arya, A. Asfaw, N. As-trakhansev, J. Atalaya, R. Babbush, D. Bacon, B. Ballard, J. C. Bardin, J. Bausch, A. Bengtsson, A. Bilmes, S. Blackwell, S. Boixo, G. Bortoli, A. Bourassa, J. Bovaird, L. Brill, M. Broughton, D. A. Browne, B. Buchea, B. B. Buckley,

- D. A. Buell, T. Burger, B. Burkett, N. Bushnell, A. Cabrera, J. Campero, H.-S. Chang, Y. Chen, Z. Chen, B. Chiaro, D. Chik, C. Chou, J. Claes, A. Y. Cleland, J. Cogan, R. Collins, P. Conner, W. Courtney, A. L. Crook, B. Curtin, S. Das, A. Davies, L. D. Lorenzo, D. M. Debroy, S. Demura, M. Devoret, A. D. Paolo, P. Donohoe, I. Drozdov, A. Dunsworth, C. Earle, T. Edlich, A. Eickbusch, A. M. Elbag, M. Elzouka, C. Erickson, L. Faoro, E. Farhi, V. S. Ferreira, L. F. Burgos, E. Forati, A. G. Fowler, B. Foxen, S. Ganjam, G. Garcia, R. Gasca, É. Genois, W. Giang, C. Gidney, D. Gilboa, R. Gosula, A. G. Dau, D. Graumann, A. Greene, J. A. Gross, S. Habegger, J. Hall, M. C. Hamilton, M. Hansen, M. P. Harrigan, S. D. Harrington, F. J. H. Heras, S. Heslin, P. Heu, O. Higgott, G. Hill, J. Hilton, G. Holland, S. Hong, H.-Y. Huang, A. Huff, W. J. Huggins, L. B. Ioffe, S. V. Isakov, J. Iveland, E. Jeffrey, Z. Jiang, C. Jones, S. Jordan, C. Joshi, P. Juhas, D. Kafri, H. Kang, A. H. Karamlou, K. Kechedzhi, J. Kelly, T. Khair, T. Khatkar, M. Khezri, S. Kim, P. V. Klimov, A. R. Klotz, B. Kobrin, P. Kohli, A. N. Korotkov, F. Kostritsa, R. Kothari, B. Kozlovskii, J. M. Kreikebaum, V. D. Kurilovich, N. Lacroix, D. Landhuis, T. Lange-Dei, B. W. Langley, P. Laptev, K.-M. Lau, L. L. Guevel, J. Ledford, K. Lee, Y. D. Lensky, S. Leon, B. J. Lester, W. Y. Li, Y. Li, A. T. Lill, W. Liu, W. P. Livingston, A. Locharla, E. Lucero, D. Lundahl, A. Lunt, S. Madhuk, F. D. Malone, A. Maloney, S. Mandrà, L. S. Martin, S. Martin, O. Martin, C. Maxfield, J. R. McClean, M. McEwen, S. Meeks, A. Megrant, X. Mi, K. C. Miao, A. Mieszala, R. Molavi, S. Molina, S. Montazeri, A. Morvan, R. Movassagh, W. Mruczkiewicz, O. Naaman, M. Neeley, C. Neill, A. Nersisyan, H. Neven, M. Newman, J. H. Ng, A. Nguyen, M. Nguyen, C.-H. Ni, T. E. O'Brien, W. D. Oliver, A. Opremcak, K. Ottosson, A. Petukhov, A. Pizzuto, J. Platt, R. Potter, O. Pritchard, L. P. Pryadko, C. Quintana, G. Ramachandran, M. J. Reagor, D. M. Rhodes, G. Roberts, E. Rosenberg, E. Rosenfeld, P. Roushan, N. C. Rubin, N. Saei, D. Sank, K. Sankaragomathi, K. J. Satzinger, H. F. Schurkus, C. Schuster, A. W. Senior, M. J. Shearn, A. Shorter, N. Shutty, V. Shvarts, S. Singh, V. Sivak, J. Skrzynski, S. Small, V. Smelyanskiy, W. C. Smith, R. D. Somma, S. Springer, G. Sterling, D. Strain, J. Suchard, A. Szasz, A. Szein, D. Thor, A. Torres, M. M. Torunbalci, A. Vaishnav, J. Vargas, S. Vdovichev, G. Vidal, B. Villalonga, C. V. Heidweiller, S. Waltman, S. X. Wang, B. Ware, K. Weber, T. White, K. Wong, B. W. K. Woo, C. Xing, Z. J. Yao, P. Yeh, B. Ying, J. Yoo, N. Yosri, G. Young, A. Zalcman, Y. Zhang, N. Zhu, and N. Zobrist, Quantum error correction below the surface code threshold, *Nature* **638**, 920 (2025), [arXiv:2408.13687](#).
- [31] D. Pataki and A. Pályi, Compiling the surface code to crossbar spin qubit architectures, *Physical Review B* **111**, 115307 (2025), [arXiv:2412.05425](#).
- [32] H. Leone, T. Le, S. Srikara, and S. Devitt, Resource overheads and attainable rates for trapped-ion lattice surgery, *Physical Review Research* **7**, 023088 (2025), [arXiv:2406.18764](#).
- [33] R. Babbush, C. Gidney, D. W. Berry, N. Wiebe, J. McClean, A. Paler, A. Fowler, and H. Neven, Encoding electronic spectra in quantum circuits with linear T complexity, *Physical Review X* **8**, 041015 (2018), [arXiv:1805.03662](#).
- [34] Z. He, A. Cowtan, D. J. Williamson, and T. J. Yoder, Extractors: QLDPC architectures for efficient Pauli-based computation, Preprint (2025), [arXiv:2503.10390](#).
- [35] D. Filippov, P. Yang, and P. Murali, Architecting distributed quantum computers: Design insights from resource estimation, Preprint (2025), [arXiv:2508.19160](#).
- [36] A. A. Kovalev and L. P. Pryadko, Quantum Kronecker sum-product low-density parity-check codes with finite rate, *Physical Review A* **88**, 012311 (2013), [arXiv:1212.6703](#).
- [37] H.-K. Lin, X. Liu, P. K. Lim, and L. P. Pryadko, Single-shot and two-shot decoding with generalized bicycle codes, Preprint (2025), [arXiv:2502.19406](#).
- [38] F. J. MacWilliams and N. J. A. Sloane, *The Theory of Error-Correcting Codes*, North-Holland Mathematical Library, Vol. 16 (Elsevier, 1977).
- [39] A. Cowtan, Z. He, D. J. Williamson, and T. J. Yoder, Parallel Logical Measurements via Quantum Code Surgery, Preprint (2026), [arXiv:2503.05003](#).
- [40] S. Bravyi and A. Kitaev, Universal quantum computation with ideal Clifford gates and noisy ancillas, *Physical Review A* **71**, 022316 (2005), [arXiv:quant-ph/0403025](#).
- [41] D. Litinski, Magic state distillation: Not as costly as you think, *Quantum* **3**, 205 (2019), [arXiv:1905.06903](#).
- [42] J. Zhang, Y.-C. Wu, and G.-P. Guo, Facilitating practical fault-tolerant quantum computing based on color codes, *Physical Review Research* **6**, 033086 (2024), [arXiv:2309.05222](#).
- [43] H. Goto, Minimizing resource overheads for fault-tolerant preparation of encoded states of the Steane code, *Scientific Reports* **6**, 19578 (2016).
- [44] T. Itogawa, Y. Takada, Y. Hirano, and K. Fujii, Efficient magic state distillation by zero-level distillation, *PRX Quantum* **6**, 020356 (2025), [arXiv:2403.03991](#).
- [45] C. Gidney, N. Shutty, and C. Jones, Magic state cultivation: Growing T states as cheap as CNOT gates, Preprint (2024), [arXiv:2409.17595](#).
- [46] K. Sahay, P.-K. Tsai, K. Chang, Q. Su, T. B. Smith, S. Singh, and S. Puri, Fold-transversal surface code cultivation, Preprint (2025), [arXiv:2509.05212](#).
- [47] S.-H. Lee, A. Li, and S. D. Bartlett, Color code decoder with improved scaling for correcting circuit-level noise, *Quantum* **9**, 1609 (2025), [arXiv:2404.07482](#).
- [48] In determining the required value of  $r$ , we upper bound the logical error rate for a logical measurement with  $r < d_t$  rounds by  $(p_L)^{\frac{d_t}{r}}$ , where  $p_L$  is the logical error rate for  $d_t$  rounds presented in Table III, which is equivalent to the worst-case assumption that all errors contributing to  $p_L$  are time-like.
- [49] A. R. O'Rourke and S. Devitt, Compare the pair: Rotated versus unrotated surface codes at equal logical error rates, *Physical Review Research* **7**, 033074 (2025), [arXiv:2409.14765](#).
- [50] S. Vittal, A. Javadi-Abhari, A. W. Cross, L. S. Bishop, and M. Qureshi, Flag-proxy networks: Overcoming the architectural, scheduling and decoding obstacles of quantum LDPC codes, in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)* (2024) pp. 718–734, [arXiv:2409.14283](#).
- [51] N. Raveendran and B. Vasić, Trapping sets of quantum LDPC codes, *Quantum* **5**, 562 (2021), [arXiv:2012.15297](#).
- [52] A. Paetznick and K. M. Svore, Repeat-until-success: Non-deterministic decomposition of single-qubit unitaries, *Quantum Information and Computation* **14**, 1277 (2014), [arXiv:1311.1074](#).
- [53] A. Bocharov, M. Roetteler, and K. M. Svore, Efficient synthesis of universal repeat-until-success circuits, *Physical Review Letters* **114**, 080502 (2015), [arXiv:1404.5320](#).

- [54] M. Ekerå and J. Håstad, Quantum algorithms for computing short discrete logarithms and factoring RSA integers, in *Post-Quantum Cryptography*, Vol. 10346 (Springer International Publishing, Cham, 2017) pp. 347–363, [arXiv:1702.00249](#).
- [55] C. Chevnard, P.-A. Fouque, and A. Schrottenloher, Reducing the number of qubits in quantum factoring, in *Advances in Cryptology – CRYPTO 2025*, edited by Y. Tautman Kalai and S. F. Kamara (Springer Nature Switzerland, Cham, 2025) pp. 384–415.
- [56] C. Jones, Low-overhead constructions for the fault-tolerant Toffoli gate, *Physical Review A* **87**, 022328 (2013), [arXiv:1212.5069](#).
- [57] D. Litinski, How to compute a 256-bit elliptic curve private key with only 50 million Toffoli gates, Preprint (2023), [arXiv:2306.08585](#).
- [58] M. E. Beverland, P. Murali, M. Troyer, K. M. Svore, T. Hoefler, V. Kliuchnikov, G. H. Low, M. Soeken, A. Sundaram, and A. Vashillo, Assessing requirements to scale to practical quantum advantage, Preprint (2022), [arXiv:2211.07629](#).
- [59] N. Baspin, L. Berent, and L. Z. Cohen, Fast surgery for quantum LDPC codes, Preprint (2025), [arXiv:2510.04521](#).
- [60] S. Aaronson and D. Gottesman, Improved simulation of stabilizer circuits, *Physical Review A* **70**, 052328 (2004), [arXiv:quant-ph/0406196](#).